



كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING



الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

SYRIAN SERVICES HUB

**A senior 2 project report - submitted to complete the
requirements for obtaining a Bachelor's Degree in
informatics engineering**

Prepared by

Lilas Nakhal

Supervised by

Eng.Amer Khorheed

Academic Year

2026



الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY



كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

منصة السورية الذكية

مشروع تخرج 2 - مُقدّم لاستكمال متطلبات الحصول على درجة
البكالوريوس في هندسة المعلوماتية
قسم البرمجيات ونظم المعلومات

إعداد

ليلاس محمد سامر نخال

إشراف

م. محمد عامر خورشيد

العام الدراسي

2026

SUPERVISION CERTIFICATION

Supervisor Certification I Certify that the preparation of this project entitled

[**Syrian Services Hub**]

Prepared by

[**Lilas Nakhal**]

Were made under our supervision at Faculty of Informatics Engineering in partial Fulfillment
of the Requirements for the Degree of Bachelor of Software and Information system
Engineering

Name:

Name:

Signature:

Signature:

Abstract

Modern service platforms are no longer limited to displaying services; they are expected to understand user needs and guide them toward the right solution quickly and intelligently. From this perspective, **Syrian Services Hub** introduces an integrated digital platform that connects users with service providers in an organized, reliable, and user-friendly environment.

The system allows users to create accounts, browse services, search for service providers, request existing services, or post custom service request advertisements when the needed service is not currently available. Users can also receive offers from providers, communicate through the internal chat system, and rate services after completion.

A key feature of the project is the integration of artificial intelligence. Users can describe their problem or need in natural language, and the system suggests suitable solutions while recommending relevant service providers from within the platform. This makes the process of finding the right service smarter and easier.

Service providers can manage their professional profiles, add services, submit offers on user requests, and track their activity through a dashboard that displays statistics, earnings, and ratings. The admin side also provides tools for managing complaints, verification requests, accounts, and platform analytics.

As a result, is not just a service-request platform, but an intelligent digital ecosystem that combines requesting, offering, communication, evaluation, administration, and AI-based recommendations in one place.

In addition to the AI recommendation feature, includes a separate Retrieval-Augmented Generation (RAG) capability named Ask Assistant. An authenticated user can ask a factual question about a selected provider or service, and the system retrieves only active, authorized provider-owned knowledge sources before generating a grounded answer with source citations. When the available evidence is insufficient, the assistant returns a clear insufficient-evidence response rather than inventing information

ملخص

لم تعد منصات الخدمات الحديثة تكتفي بعرض الخدمات فقط، بل أصبحت مطالبة بفهم حاجة المستخدم Syrian Services وتوجيهه نحو الحل المناسب بسرعة وذكاء. ومن هذا المنطلق، يقدم مشروع منصة رقمية متكاملة تربط بين المستخدمين ومزودي الخدمات ضمن بيئة منظمة، موثوقة، وسهلة الاستخدام Hub.

يتيح النظام للمستخدم إنشاء حساب، تصفح الخدمات، البحث عن مزودي الخدمات، طلب خدمة موجودة، أو نشر إعلان عن خدمة يحتاجها في حال لم تكن متوفرة على المنصة. كما يستطيع المستخدم استقبال العروض من مزودي الخدمات، التواصل معهم عبر نظام المحادثة، وتقييم الخدمة بعد الانتهاء منها.

يتميز المشروع دمج الذكاء الاصطناعي، حيث يستطيع المستخدم كتابة مشكلته أو حاجته بلغة طبيعية، ليقوم النظام باقتراح حلول مناسبة وترشيح مزودي خدمات من داخل المنصة، مما يجعل تجربة البحث عن الخدمة أكثر ذكاءً وسهولة.

أما مزود الخدمة، فيستطيع إدارة ملفه المهني، إضافة خدماته، تقديم عروض على طلبات المستخدمين، ومتابعة نشاطه من خلال لوحة تحكم تعرض الإحصائيات والأرباح والتقييمات. كما يوفر النظام للأدمن أدوات لإدارة الشكاوى، طلبات التوثيق، الحسابات، ومراقبة أداء المنصة من خلال لوحة تحكم تحليلية.

وبذلك لا تعد مجرد منصة لطلب الخدمات، بل منظومة ذكية تجمع بين الطلب، العرض، التواصل، التقييم، الإدارة، والتوصية الذكية في مكان واحد.

، تتضمن منصة ميزة مستقلة تعتمد على تقنية التوليد AI Recommendation إضافةً إلى ميزة تتيح هذه الميزة للمستخدم المسجل طرح سؤال واقعي حول مزود خدمة أو . (RAG) المعزز بالاسترجاع خدمة محددة

Contents

Section 1 Project Introduction	14
Chapter One: Introduction	15
1.1 Project Overview and Background.....	15
1.2 Problem Statement and Significance	15
1.3 Project Objectives	16
1.4 Scope, Constraints, and Assumptions.....	16
1.5 Project Contribution	17
1.6 Research Methodology	18
1.7 Document Structure	18
1.8 Definitions and Key Concepts	19
Chapter Two: Project Development and Management Methodology	20
2.1 Selected Development Methodology and Justification	20
2.2 Project Timeline and Work Plan	20
2.3 Risk Management	21
2.4 Team Roles and Task Distribution	22
2.5 Quality Management in the Project	23
2.6 Project Management Tools Used.....	23
2.7 Monitoring and Continuous Evaluation Mechanisms	24
2.8 Software Configuration Management	24
Section Two: Theoretical Study and Analysis	25
Chapter Three: Literature Review and Comparative Study	26
3.1 Related Work in Smart and Similar Service Platforms	26
3.2 Modern Technologies and Frameworks in the Domain	26
3.3 Comparative Analysis of Existing Platforms	27
3.4 Research Gap and Proposed Innovation	28
3.5 Theoretical Background and Reference Foundations.....	29
3.6 Modern and Future Trends in the Domain	29
Chapter Four: Requirements Analysis and Modeling	31
3.7 Retrieval-Augmented Generation for Provider Question Answering	31
4.1 Feasibility Study	31

4.2 Requirements Gathering.....	32
4.3 Stakeholders and Needs Analysis.....	32
4.4 Functional Requirements.....	33
4.5 Non-Functional Requirements.....	39
4.6 Requirements Prioritization and Management	40
4.7 UML Requirements Modeling.....	41
4.8 Modeling According to the Selected Methodology.....	55
4.9 Requirements Analysis and Documentation.....	57
Section Three: Design and Architecture.....	65
Chapter Five: System Architecture and Design	66
5.1 High-Level System Design.....	66
5.2 Database Design	70
5.3 External API Design.....	74
5.4 Security and Access Design	75
Section Four: Implementation and Testing	77
Chapter Six: Implementation Environment and Tools	78
6.1 Software Tools and Libraries Used	78
6.2 Development Environment and Project Structure	79
6.3 Programming Languages and Frameworks	80
6.4 Database Server and Operating System.....	81
6.5 Dependencies.....	81
6.6 Version Control and Source Management Tools	82
6.7 Continuous Integration and Deployment Tools.....	83
6.8 Testing Environment	83
Chapter Seven: Software Implementation.....	85
7.1 Project Structure and File Organization	85
7.2 Implementation of Major Functional Modules.....	86
7.3 API Implementation	87
Request Processing Flow.....	87
1. Main API Route.....	88
2. Service Creation Endpoint.....	88

4. API View Implementation.....	89
5. Business Rule and Database Creation	90
6. Example Request Body	90
7. Example Response Body	90
7.4 Implementation of the Data-Access Layer	91
7.5 Implementation of the Business-Logic Layer	91
7.6 User Interface Implementation	92
7.7 Implementation of the Intelligent Recommendation-Oriented Functionality	93
7.8 Implementation of Ask Assistant (RAG)	94
Chapter Eight: Testing and Evaluation.....	96
8.1 Test Plan	96
8.2 Types of Testing	97
8.3 Test Diagrams	98
8.4 Test Results and Analysis.....	99
8.5 Quality Metrics	109
8.6 Usability Testing.....	110
8.7 Evaluation Summary	111
Section Five: Results and Recommendations.....	112
Chapter Nine: Results and Analysis	113
9.1 System Results.....	113
9.2 Comparative Analysis with Existing Systems.....	114
9.3 Achievement of Project Objectives	115
9.4 Strengths and Limitations	116
9.5 Key Performance Indicators	116
Chapter Ten: Conclusion and Recommendations	118
10.1 Project Summary and Main Achievements	118
10.2 Difficulties and Challenges	119
10.3 Future Improvements.....	120
10.4 Recommendations	121
10.5 Closing Statement.....	122
Section Six: Appendices and References	123

TABLE OF TABLES

table 1 Risk Register	17
table 2 Comparative Analysis of Reference Platforms	21
table 3 Stakeholders and Needs Analysis	23
table 4 Functional Requirements	24
table 5 Non-Functional Requirements	30
table 6 Requirements Prioritization	31
table 7 Use Case Descriptions- Submit Complaint	32
table 8 Use Case Descriptions accept or reject offer	33
table 9 Use Case Descriptions submit offer on request	33
table 10 Use Case Descriptions add services	34
table 11 Use Case Descriptions post new request	34
table 12 Increment Plan and User Stories	41
table 13 Story Mapping	41
table 14 Requirements Traceability Matrix	42
table 15 Initial Test Cases-add services	43
table 16 Initial Test Cases send offer	45
table 17 Initial Test Cases post new request	46
table 18 Initial Test Cases submit offer on request	Error! Bookmark not defined.
Table 19 Initial Test Cases Create User Account	Error! Bookmark not defined.
table 20 Architecture Layer Summary	50
table 21 Core Relational Schema of Syrian Services Hub	52
table 22 Main Database Constraints	54
table 23 Representative API Endpoints	56
table 24 Main software tools and libraries used in the implementation environment	59
table 25 High-level implementation environment structure	60
table 26 Programming languages and frameworks used by the project	61
table 27 Database and execution environment summary	61
table 28 Main dependency groups used by the project	62
table 29 Version control and source-management tools	63
table 30 CI/CD status within the current project scope	63
table 31 Testing environment elements used during implementation	64
table 32 High-level repository structure of Syrian Services Hub	65
table 33 Main implementation modules and their responsibilities	67
table 34 Representative API functions in the implemented system	Error! Bookmark not defined.
table 35 Data-access responsibilities in the implementation	71
table 36 Representative business-logic responsibilities	71
table 37 Implementation positioning of the recommendation-oriented feature	74
table 38 Test plan overview	75

table 39	Types of testing applied to Syrian Services Hub	76
table 40	Main test cases and validation results add services	78
table 41	Main test cases and validation results submit offer	80
table 42	Main test cases and validation results post new request	81
table 43	Main test cases and validation results submit offer	Error! Bookmark not defined.
Table 44	Main test cases and validation results Create User Account	Error! Bookmark not defined.
table 45	Quality metrics used in the evaluation	83
table 46	Usability review observations	84
table 47	Comparative analysis between Syrian Services Hub and selected reference platforms.	88
table 48	Achievement of the main project objectives.	89
table 49	Summary of project strengths and limitations.	90
table 50	Suggested KPIs for evaluating Syrian Services Hub.	90
table 51	Summary of main project achievements.	Error! Bookmark not defined.
table 52	Main project challenges and responses.	Error! Bookmark not defined.
table 53	Suggested future improvements.	Error! Bookmark not defined.
table 54	Final recommendations.	Error! Bookmark not defined.
table 55	Appendix Overview	Error! Bookmark not defined.

TABLE OF FIGURES

Figure 1 Gantt Chart	Error! Bookmark not defined.
Figure 2 Main Use Case Diagram for Syrian Services Hub	95
Figure 3 use case request an existing service	95
Figure 4 use case post new request	96
Figure 5 use case submit an offer	96
Figure 6 Post Service Request	35
Figure 7 Activity Diagram - Submit Offer	36
Figure 8 Activity Diagram submit complaint	36
Figure 9 Activity Diagram add services	Error! Bookmark not defined.
Figure 10 Activity Diagram review incoming offer	Error! Bookmark not defined.
Figure 11 Sequence Diagram - Submit Offer	37
Figure 12 Sequence Diagram - Submit Offer	38
Figure 13 Sequence Diagram - add services	39
Figure 14 Sequence Diagram -post new request	40
Figure 15 Sequence Diagram -review incoming offer	Error! Bookmark not defined.
Figure 16 : Sequence Diagram submit complaint	Error! Bookmark not defined.
Figure 18 System Block Diagram of Syrian Services Hub	50
Figure 19 Architecture of Syrian Services Hub	Error! Bookmark not defined.
Figure 20 ER Diagram of Syrian Services Hub	52
Figure 21 Placeholder for the project structure	66
Figure 22 Placeholder for a main marketplace interface screenshot.	73
Figure 23 service-request interface screenshot	73
Figure 24 dashboard of Syrian Services Hub.	88

Section 1

Project Introduction

Chapter One: Introduction

This chapter introduces the proposed platform, explains the practical problem it addresses, defines the main objectives of the project, and outlines the intended scope and contribution of the system.

1.1 Project Overview and Background

Syrian Services Hub is a smart digital service marketplace connecting consumers with service providers through a structured platform. It supports visitors, users, providers, and administrators, with functions including service browsing, requests, offers, messaging, verification, and administration. The platform also includes **AI Recommendation** for suggesting solutions and providers, and **Ask Assistant** for provider-specific, evidence-based question answering.

1.2 Problem Statement and Significance

Consumers often struggle to find reliable local service providers because services are scattered across social media, personal contacts, and unstructured channels. Providers also face difficulties reaching suitable customers and presenting their services professionally. Syrian Services Hub addresses this gap through a unified marketplace with structured service discovery, requests, offers, communication, and intelligent recommendation support.

1.3 Project Objectives

The project aims to create an accessible and organized marketplace that simplifies service discovery, requests, provider offers, and communication. It also strengthens trust through ratings, complaints, verification, and administrative control. Intelligent support is provided through **AI Recommendation** for service/provider matching and **Ask Assistant** for grounded, citation-based answers from authorized provider knowledge.

1.4 Scope, Constraints, and Assumptions

The project covers service browsing, requests, offers, provider profiles, administrative functions, recommendation support, and provider-grounded question answering. Ask Assistant includes knowledge ingestion, chunking, embeddings, retrieval, conversation history, and citations. The main constraints are limited academic time and scope, while users are assumed to have internet access and basic digital literacy.

1.5 Project Contribution

Syrian Services Hub contributes a **hybrid local marketplace** combining direct service browsing with open service requests and intelligent decision support. AI Recommendation helps users identify suitable solutions and providers, while Ask Assistant introduces grounded, traceable question answering using provider-owned knowledge.

1.6 Research Methodology

The project follows an **engineering and descriptive methodology**. The engineering aspect covers requirements, architecture, implementation, and evaluation, while the descriptive aspect analyzes current service-discovery challenges and explains how a structured smart platform can address them.

1.7 Document Structure

The thesis progresses from project background and management methodology to related work, requirements analysis, system design, implementation, testing, results, conclusions, future improvements, references, and appendices.

1.8 Definitions and Key Concepts

Key concepts include the **service marketplace, visitor, registered user, service provider, administrator, recommendation system, service-demand advertisement, functional and non-functional requirements, RAG, knowledge sources, knowledge chunks, and assistant citations**. These concepts define the platform's operational and intelligent foundations.

Chapter Two: Project Development and Management

Methodology

This chapter explains the development methodology adopted for the project, the overall work plan, the main risks and quality considerations, and the mechanisms used to monitor progress and maintain consistency throughout the project lifecycle. It also clarifies how the project was organized from a management perspective in order to support both the technical implementation of the platform and the preparation of its academic documentation.

2.1 Selected Development Methodology and Justification

Syrian Services Hub follows an **Incremental Development** methodology due to its multiple interconnected modules. The system is developed in manageable increments, allowing each group of features to be designed, implemented, tested, and refined before further functionality is added. This approach supports clear prioritization, controlled integration, and effective delivery within the academic timeframe.

2.2 Project Timeline and Work Plan

The project is organized into five main phases: **requirements analysis, system modeling, design, implementation, and testing/documentation**. Core modules are developed progressively, followed by validation and final thesis preparation. This structure ensures steady progress while keeping the project manageable and aligned with academic deadlines.

Updated Project Gantt Chart - Sprint 6 with Lilas

Sprint 6 retains Lilas only. Project schedule remains unchanged and ends on 03/07/2026.

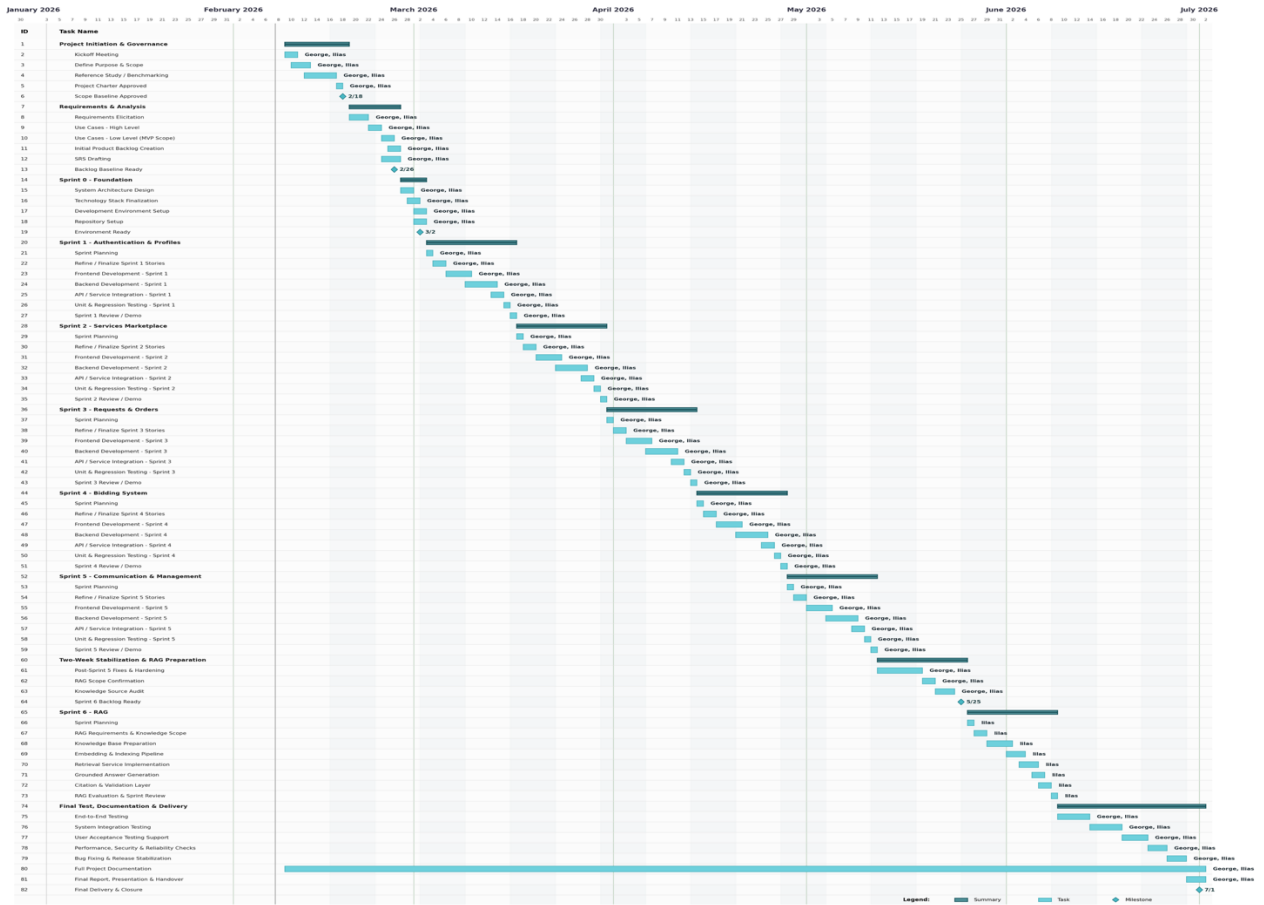


Figure 1 Gantt Chart

2.3 Risk Management

Key risks include changing requirements, implementation complexity, limited development time, and inconsistencies between analysis, design, and implementation. These risks are reduced through incremental development, regular requirement reviews, prioritization of essential features, and structured documentation.

2.4 Team Roles and Task Distribution

Project tasks are distributed across **requirements analysis, system design, modeling, implementation, testing, and documentation**. The workflow also considers the integration of the recommendation component with the platform's core services, ensuring balanced attention to both technical development and academic requirements.

2.5 Quality Management in the Project

Quality management focuses on **consistency, reliability, clarity, and alignment** between requirements, design, implementation, and documentation. Regular reviews of use cases, system models, and functional relationships help reduce ambiguity and ensure that all platform components, including the recommendation feature, contribute meaningfully to the system.

2.6 Project Management Tools Used

Project management relies on structured documentation, task tracking, organized file management, and version control. These practices help coordinate development and documentation activities, preserve consistency, and reduce the risk of lost or conflicting work.

2.7 Monitoring and Continuous Evaluation Mechanisms

Continuous evaluation is performed through regular reviews of requirements, use cases, system structure, and implementation progress. This allows inconsistencies, missing requirements, and unnecessary complexity to be identified early and corrected before later development stages.

2.8 Software Configuration Management

Software Configuration Management ensures **version control, traceability, and consistency** across project files, documentation, models, and source code. Organized configuration practices reduce duplication, prevent accidental data loss, and ensure that development remains aligned with the latest approved project version.

table 1 Risk Register

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy
R-01	Requirements may change or become clearer during development.	Medium	High	Use iterative review, keep requirements traceable, and prioritize core features.
R-02	Time limitations may reduce implementation or testing depth.	High	High	Apply phased work planning and focus first on must-have features.

R-03	Recommendation functionality may become too complex.	Medium	Medium	Keep the first version focused on problem-based suggestions and provider matching.
R-04	Inconsistency between requirements, design, and implementation.	Medium	High	Use RTM, use cases, and continuous documentation review.
R-05	Data or diagram updates may affect documentation accuracy.	Medium	Medium	Review models and tables before final submission.

Section

Two: Theoretical

Study and Analysis

Chapter Three: Literature Review and Comparative Study

This chapter establishes the theoretical and comparative foundation of **Syrian Services Hub** by reviewing similar service marketplaces, their operational models, and the features that influenced the proposed system. It also introduces the hybrid marketplace concept and supports later requirements and design decisions.

3.1 Related Work in Smart and Similar Service Platforms

Syrian Services Hub belongs to the domain of digital service marketplaces connecting providers and customers through structured workflows. Benchmark platforms include **Khamsat**, **Mostaq**, **Fiverr**, **Upwork**, **Freelancer**, and **PeoplePerHour**, representing two main models: **ready-made services** and **open service requests with bidding**. The proposed system combines key features from these platforms while adapting them to its local and academic context.

3.2 Modern Technologies and Frameworks in the Domain

Modern service platforms rely on scalable web technologies and intelligent support features. **React** provides responsive interfaces, **Django** supports backend logic and role-based workflows, **PostgreSQL** manages relational data, and **Postman** supports API testing. Recommendation mechanisms further improve service discovery and user decision-making.

3.3 Comparative Analysis of Existing Platforms

Existing platforms provide different strengths. **Khamsat** and **Fiverr** focus on ready-made services, while **Mostaq** emphasizes open requests and provider offer. **Upwork** combines project workflows with messaging, verification, and trust mechanisms, while **Freelancer** and **PeoplePerHour** provide useful bidding and hybrid interaction models. Based on this analysis, Syrian Services Hub adopts a **hybrid marketplace model** combining ready-made services and open service requests in one platform.

table 2Comparative Analysis of Reference Platforms

Platform	Dominant Model	Main Reference Components	Design Role in Syrian Services Hub
Khamsat	Ready-made services	Service, seller profile, ratings, messages	Reference for service listing, browsing, and trust indicators.
Mostaqi	Open requests and offers	Project/request, offers, provider selection	Reference for service-demand advertisements and competitive offers.
Fiverr	Gigs / packaged services	Gig page, packages, add-ons	Reference for structured service presentation and clear scope.
Upwork	Hybrid marketplace	Project Catalog, proposals, direct messages, verification	Reference for combining service catalog, requests, messaging, and trust mechanisms.
Freelancer	Bidding and dashboards	Projects, bids, milestones, dashboards	Reference for offer status, provider activity, and dashboard tracking.
PeoplePerHour	Offers and custom projects	Offer, proposal, escrow logic	Reference for coexistence of ready-made offers and custom requests.

3.4 Research Gap and Proposed Innovation

Existing marketplaces provide strong features, but these are often distributed across different platforms. Syrian Services Hub addresses this gap through a **hybrid marketplace model** that combines ready-made services, open requests, provider comparison, and recommendation support. Its innovation lies in integrating proven marketplace practices into a locally adapted, decision-oriented platform.

3.5 Theoretical Background and Reference Foundations

Syrian Services Hub is based on **digital marketplace principles**, where users and providers interact through structured workflows for services, requests, offers, communication, trust, and administration. Its hybrid model combines direct service publishing with open service requests, while intelligent recommendations support better user decisions.

3.6 Modern and Future Trends in the Domain

Modern service marketplaces are becoming more **intelligent, personalized, and user-centered**. Major trends include recommendation systems, responsive interfaces, stronger verification, transparent ratings, complaint management, and improved platform governance. Syrian Services Hub reflects these trends through its hybrid structure, intelligent features, and trust mechanisms.

3.7 Retrieval-Augmented Generation for Provider Question Answering

Retrieval-Augmented Generation (RAG) combines information retrieval with generative AI to produce grounded and traceable answers.

In Syrian Services Hub, **Ask Assistant** retrieves authorized provider knowledge and generates evidence-based responses with citations. It remains separate from **AI Recommendation**, which focuses on matching users with suitable services and providers.

Chapter Four: Requirements Analysis and Modeling

4.1 Feasibility Study

Syrian Services Hub is feasible from **technical, practical, and academic perspectives**. Technically, its core functions can be implemented using modern web technologies supporting role-based access, database-driven workflows, and intelligent features. Practically, it addresses the need for an organized local marketplace connecting users and service providers. Academically, the system provides sufficient scope for requirements analysis, modeling, design, implementation, and testing using software engineering principles.

4.2 Requirements Gathering

System requirements were identified through analysis of the project objectives, stakeholder roles, marketplace workflows, and benchmark platforms. The resulting requirements cover **accounts, provider profiles, services, service requests, offers, messaging, ratings, complaints, verification, dashboards, provider comparison, and recommendation support**.

4.3 Stakeholders and Needs Analysis

The main stakeholders are **visitors, registered users, service providers, and administrators**. Each role has specific responsibilities and expectations that are reflected in the system requirements and supported workflows.

table 3 Stakeholders and Needs Analysis

Stakeholder	Main Needs	System Support
Visitor	Browse services before registration and understand available providers.	Public service browsing and basic service/provider information.
Registered User	Request services, post service-demand advertisements, receive offers, compare providers, and get recommendations.	Service request workflow, offer review, messaging, ratings, and recommendation page.
Service Provider	Publish services, manage profile, submit offers, communicate with users, and monitor activity.	Provider profile, service management, offer management, chat, dashboard, and verification request.

Administrator	Control the platform, manage users, review complaints, approve or revoke verification, and monitor statistics.	Admin dashboard, complaint management, verification management, and account control.
Registered User - Ask Assistant	Ask factual questions about a selected provider or service and understand the evidence behind the answer.	Authenticated question submission, grounded answer, source citations, and conversation history.
Service Provider - Knowledge Sources	Upload, list, archive, and reindex customer-facing knowledge sources while controlling what customers may retrieve.	Knowledge-source management, ingestion status, visibility control, and reindex operations.

4.4 Functional Requirements

The functional requirements define the core behaviors that Syrian Services Hub must provide. They represent the main system capabilities required to support both direct service browsing and open service-request workflows. The table below summarizes the key functional requirement groups used in the project. The recommendation requirement and the ability of the administrator to revoke provider verification are included explicitly to keep the requirements aligned with the current system scope.

table 4 Functional Requirements

Requirement ID	Title	Type	Priority	Owner	Description	Acceptance Criteria
AUTH-001	Create User Account	Functional	High	Visitor	Allows a visitor to create a new user account by entering name, email, and password.	A new record is created in the Users table, and the email address and phone number must be unique.
AUTH-002	Create Service Provider Account	Functional	High	Visitor	Allows a visitor to register as a service provider by entering name, email, password, and category.	An account with the service provider role is created, and a linked service provider profile is created.
AUTH-003	Login	Functional	High	User / Provider / Admin	Allows a user, service provider, or admin to log in using email and password.	The credentials are validated, and a valid login session is created.
AUTH-004	Logout	Functional	Medium	User / Provider / Admin	Allows a user, service provider, or admin to log out of the system.	The current user session is ended.
AUTH-005	Change Password	Functional	High	User / Provider	Allows a user or service provider to change the password after entering the	The password is updated in the database after verifying the

					current password and the new password.	current password.
PROF-001	Edit Personal Information	Functional	High	User / Provider	Allows the user to edit personal information such as name, email, phone number, and location.	The user data is successfully updated in the database.
PROV-002	Edit Service Provider Information	Functional	High	Provider	Allows the service provider to edit professional information such as skills, description, and location.	The service provider data is updated in the Service Providers table.
PROV-003	View Service Providers	Functional	High	User / Visitor	Allows the user to browse the list of registered service providers and view basic information such as name, rating, and provided services.	Service provider data is retrieved from the database and displayed to the user.
PROV-004	Compare Service Providers	Functional	Medium	User	Allows the user to select multiple service providers and compare information such as price, rating, execution time, and number of provided services.	A comparison table is displayed containing the selected service providers' information.
SERV-001	Add Service	Functional	High	Provider	Allows the service provider to add a new service including service name, description, price, and execution duration.	A new service record linked to the service provider is created.
SERV-002	Edit Service	Functional	High	Provider	Allows the service provider to edit the data of a service that belongs to them.	The service data is updated in the database.
SERV-003	Delete Service	Functional	High	Provider	Allows the service provider to delete a service they added.	The service is deleted or deactivated in the database.
SERV-004	View Services	Functional	High	User / Visitor	Allows users to browse the available services on the platform.	Services are retrieved from the database and displayed to the user.
SERV-005	Search for a Service	Functional	High	User / Visitor	Allows the user to search for services using keywords.	Services matching the search query are displayed.
SERV-006	Filter Services	Functional	High	User / Visitor	Allows the user to filter services by price, rating, or verified accounts.	Services matching the filter criteria are displayed.
RECO-001	Recommendation	Functional	High	System / User	Allows the system to provide helpful advice to users and recommend suitable service providers based on the selected service category, request description, provider skills, ratings, location, and service information.	The system displays useful advice and a list of suitable service providers related to the user's request or selected service category.

SERV-007	Send Request for an Existing Service	Functional	High	User	Allows the user to submit a request for an existing service that was previously added by a service provider.	A new request record is created in the database and linked to the user, service, and service provider, with the status set to Pending.
SERV-008	Send Offer on Service Request	Functional	High	Provider	Allows the service provider to respond to a service request sent by the user by sending an offer that includes price, duration, or execution details.	A new offer record linked to the service request is created, and the request status is updated to indicate that an offer has been sent.
SERV-009	Reject Service Request	Functional	High	Provider	Allows the service provider to reject a service request sent by the user.	The request status is updated in the database to Rejected.
SERV-010	Complete Service	Functional	High	Provider	Allows the service provider to complete the service after finishing its execution for the user.	The service or request status is updated in the database to Completed or Finished successfully.
ORDER-001	Track Service Request Status	Functional	High	User	Allows the user to track the status of service requests sent to service providers to know whether they are still pending, rejected, or answered with an offer.	The status of each service request is displayed correctly, such as Pending, Rejected, or Offer Sent, and the data is retrieved from the database correctly.
BID-001	Post New Service Request	Functional	High	User	Allows the user to publish a service request with a service description and expected budget.	A new service request record is created in the database.
BID-002	Submit Offer on Request Post	Functional	High	Provider	Allows the service provider to submit an offer on the service request post sent by the user.	A new offer record linked to the service request post is created and contains the offer data sent by the service provider.
BID-003	Edit Offer	Functional	Medium	Provider	Allows the service provider to edit an offer they previously submitted.	The offer data is updated in the database.
BID-004	Delete Offer	Functional	Medium	Provider	Allows the service provider to delete an offer they submitted.	The offer is deleted or deactivated in the database.
BID-005	Accept or Reject Offer	Functional	High	User	Allows the user to review the offer sent by the service provider	The selected offer status is updated to Accepted or Rejected

					and then accept or reject it.	according to the user's action.
BID-006	Track Offer Status	Functional	High	Provider	Allows the service provider to track the status of an offer they sent to know whether it was accepted, rejected, or is still waiting for the user's response.	The status of each offer is displayed correctly for the service provider, such as Pending, Accepted, or Rejected.
BID-007	Edit Service Request Post	Functional	Medium	User	Allows the user to edit a service request post they previously published before selecting a final offer.	The service request post data is successfully updated in the database.
BID-008	Delete Service Request Post	Functional	Medium	User	Allows the user to delete a service request post they previously published.	The service request post is deleted or deactivated in the database.
CHAT-001	Create Conversation	Functional	High	System	The system creates a conversation between the user and the service provider when cooperation starts between them.	A new conversation record is created in the database.
CHAT-002	Send Message	Functional	High	User / Provider	Allows the user or service provider to send a message within the conversation.	The message is saved in the Messages table.
REV-001	Rate Service	Functional	High	User	Allows the user to rate the service after completing the request.	A new rating record is created in the database.
REV-002	Add Comment within Rating	Functional	High	User	Allows the user to add a comment or text note within the rating after completing the service.	The comment is saved successfully with the rating record in the database.
REV-003	View Rating Comments	Functional	Medium	User / Provider	Allows the system to display comments linked to ratings on the ratings page or in the service provider profile.	Comments linked to ratings are retrieved from the database and displayed correctly.
COMP-001	Submit Complaint	Functional	High	User	Allows the user to submit a complaint against a service provider.	A complaint record is created with the status Open.
COMP-002	Manage Complaints	Functional	High	Admin	Allows the admin to review complaints and change their status.	The complaint status can be changed to Under Review, Rejected, or Closed.
VER-001	Request Service Provider Verification	Functional	High	Provider	Allows the service provider to submit a request to verify their account.	A new verification request is created with the status Pending.
VER-002	Accept or Reject Verification Request	Functional	High	Admin	Allows the admin to review verification requests and accept or reject them.	The verification request status is updated in the database.

ADMIN-001	Block and Unblock Accounts	Functional	High	Admin	Allows the admin to block or unblock user and service provider accounts when needed.	The account status is updated in the database to Blocked or Active according to the admin's action.
DASH-P-001	Service Provider Dashboard	Functional	High	Provider	The dashboard displays information about the service provider's activity, such as completed tasks, average rating, offers, and earnings.	Statistical data is displayed from the database.
DASH-P-002	Service Provider Earnings Dashboard	Functional	High	Provider	The service provider dashboard displays detailed earnings information, including total earnings, current month earnings, a monthly earnings chart, and average earnings per service.	Total earnings and current month earnings are displayed correctly, a monthly earnings chart is shown, and average earnings per service are calculated and displayed in the dashboard.
DASH-A-001	Admin Dashboard	Functional	High	Admin	The dashboard displays general platform statistics such as number of users and number of requests.	Data is displayed from the database.
DASH-A-002	Admin Analytics Dashboard	Functional	High	Admin	The admin dashboard displays platform activity analytics and statistics such as average offer value, offer acceptance rate, most requested category, total number of submitted offers, request distribution by category, and offer status distribution.	Average offer value, offer acceptance rate, and most requested category are displayed correctly, and charts show offer totals, request distribution by category, and offer status distribution.
UI-001	Multilingual Support	Non-Functional	Medium	System	The system supports Arabic and English languages in the user interface.	The user can change the interface language, and the text is updated across all system pages.
UI-002	Dark and Light Mode	Non-Functional	Medium	System	Allows the user to switch between dark mode and light mode in the interface.	The selected theme is applied to all system pages and the user's choice is saved.
NFR-001	Performance and Response Speed	Non-Functional	High	System	The system must respond quickly to user actions while browsing services, searching, sending requests, and submitting offers.	The response time for basic operations must not exceed 3 seconds under normal conditions.

NFR-002	Availability	Non-Functional	High	System	The system must be available to users, service providers, and admins most of the time without interruption.	The system must achieve at least 99% availability during the operating period.
NFR-003	Security	Non-Functional	High	System	The system must protect user data, passwords, and sessions from unauthorized access.	Passwords are encrypted, access permissions are verified, and unauthorized access to pages and data is prevented.
NFR-004	Usability	Non-Functional	Medium	System	The system interfaces must be clear and easy to use for all types of users.	The user can perform basic operations such as registration, search, and sending requests without needing additional explanation.
NFR-005	Scalability	Non-Functional	Medium	System	The system must support an increase in the number of users, services, and requests without major performance degradation.	The system can handle a noticeable increase in users and requests while maintaining acceptable performance.
NFR-006	Compatibility with Different Devices	Non-Functional	Medium	System	The system must work correctly on desktops, mobile phones, and tablets.	System pages are displayed correctly on different screen sizes.
RAG-001	Ask Assistant	Functional	High	User	Allows an authenticated user to ask a factual question about a selected provider or service. The system retrieves relevant evidence only from active, authorized provider-owned knowledge sources and generates a grounded answer.	The system displays an answer with source citations when evidence is sufficient. If evidence is insufficient, it returns a clear insufficient-evidence response and does not invent facts.
RAG-002	Manage Knowledge Sources	Functional	High	Provider	Allows a provider to upload, list, archive, and reindex customer-facing knowledge sources, while authorized administration can review ingestion status and restricted records.	Source ownership, visibility, ingestion status, and indexing state are stored. Private or archived sources are excluded from customer retrieval.

4.5 Non-Functional Requirements

The non-functional requirements define the quality attributes that the platform must maintain. They support usability, security, performance, availability, scalability, and compatibility across different devices. These requirements guide later design and testing decisions.

table 5 Non-Functional Requirements

Req. ID	Quality Attribute	Priority	Requirement Description	Acceptance Criteria
NFR-001	Performance	High	The system should respond efficiently during browsing, searching, requesting, and offer submission.	Core operations should respond within an acceptable time under normal conditions.
NFR-002	Availability	High	The platform should remain available for users, providers, and administrators most of the time.	The system should be accessible during normal operation with minimal interruption.
NFR-003	Security	High	The system should protect accounts, sessions, passwords, and role-based access.	Passwords are protected, unauthorized access is blocked, and sensitive pages require permission.
NFR-004	Usability	Medium	The platform should be clear and easy to use for all user roles.	Users can complete essential operations without unnecessary complexity.
NFR-005	Scalability	Medium	The system should support growth in users, services, requests, and offers.	The design supports additional records and future feature expansion.
NFR-006	Compatibility	Medium	The interface should work correctly on different screen sizes and devices.	Pages remain readable and usable on desktop, tablet, and mobile screens.
NFR-007	Groundedness and Privacy	High	Ask Assistant must answer only from authorized active evidence, preserve provider ownership and visibility rules, cite supporting sources, and resist instructions contained inside retrieved documents.	Every supported factual answer includes citations; private, archived, or unrelated sources are excluded; and unsupported questions produce an insufficient-evidence response.

4.6 Requirements Prioritization and Management

The project requirements are prioritized to distinguish the core marketplace functions from supporting and value-adding functions. The MoSCoW method is used to organize requirements into Must Have, Should Have, Could Have, and Won't Have for the current scope. This helps maintain implementation focus and reduces the risk of unnecessary scope expansion.

table 6 Requirements Prioritization

Priority	Included Requirements	Reason
Must Have	Authentication, service management, service requests, offers, messaging, complaint handling, verification, recommendation feature.	These requirements form the operational core of the platform.
Should Have	Ratings, comments, dashboards, provider comparison, search and filtering improvements.	They improve trust, decision-making, and user experience.
Could Have	Advanced analytics, expanded recommendation criteria, automated moderation, notification enhancements.	They add value but can be implemented progressively.
Won't Have Now	Online payment integration, mobile application, advanced AI chatbot, external platform integration.	They are outside the current academic project scope.
Must Have	Ask Assistant (RAG), provider knowledge-source management, permission-aware retrieval, answer citations, and insufficient-evidence handling.	These requirements are necessary for safe, traceable, evidence-based customer question answering.

4.7 UML Requirements Modeling

UML requirements modeling provides a visual representation of the system's main behaviors and interactions. Since Syrian Services Hub includes several actors and interconnected workflows, UML models are necessary to clarify how users, providers, visitors, and administrators interact with the system. The most important UML artifacts in this chapter include use case diagrams, use case descriptions, activity diagrams, and sequence diagrams.

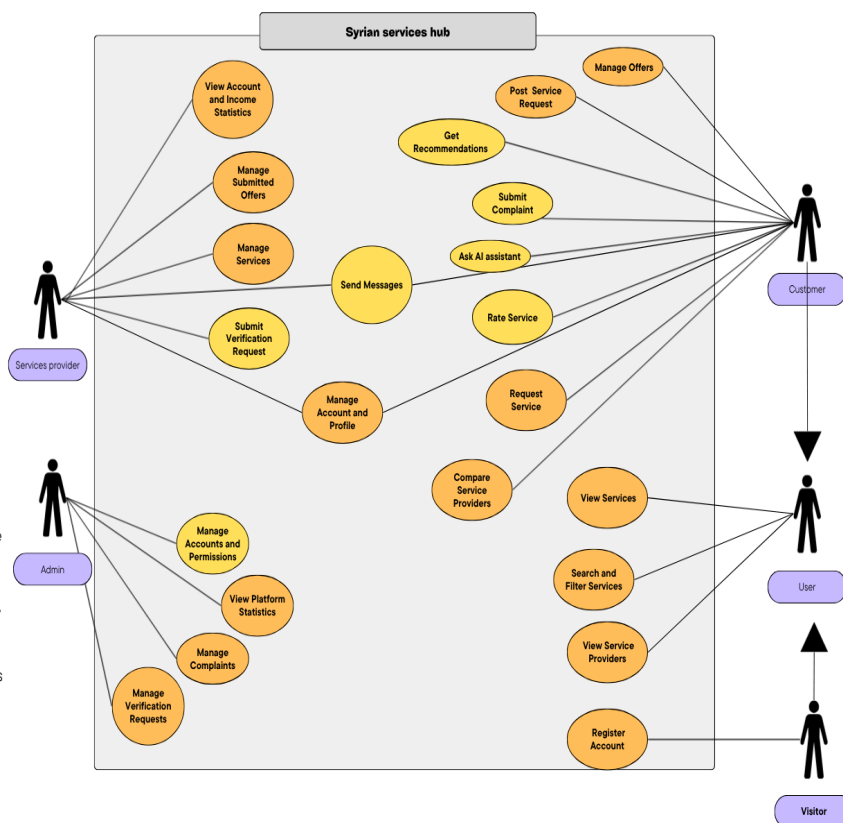
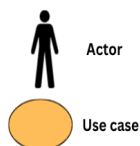
4.7.1 Main Use Case Diagrams

The main use case diagram should present the primary actors and the most important system interactions. It should focus on the functional core instead of listing every minor operation. For this project, the diagram should include Visitor, User, Service Provider, and Administrator as the main actors. The key use cases should include browsing services, posting a service request,

submitting an offer, accepting or rejecting an offer, managing services, managing complaints, managing verification, revoking verification, and getting problem-based recommendations.

UML Use Case Diagram

This diagram represents the High-Level Use Case Diagram for the service marketplace system. It illustrates the main interactions between the system and its primary actors. The actors include Visitor, who can browse the platform and create an account; User, who requests services; Provider, who offers services and submits offers; and Admin, who manages and monitors the platform.



4.7.2 Use Case Descriptions

Use case descriptions explain the most important interactions in a structured textual form. Each description identifies the actor, objective, preconditions, main flow, exceptional flow, and postconditions. The table below summarizes the main use cases selected for the analysis chapter.

table 7 Use Case Descriptions- Submit Complaint

Field	Description
Use Case ID	COMP-001
Use Case Name	Submit Complaint
Actors	User

Goal	Enable the user to submit a complaint against a service provider.
Preconditions	1. The user is logged in. 2. There has been a previous interaction with the service provider.
Main Flow	1. System: Displays the “Submit Complaint” option and the complaint form. 2. User: Enters the complaint details and clicks “Submit”. 3. System: Verifies that the complaint is not empty, then creates a new complaint record with the status “Open”. 4. User: Returns to the previous page or the complaints page. 5. System: Displays the new complaint status and shows a success message.
Exceptional Flows	EF1: The complaint details are empty or invalid.
Postconditions	A complaint record is created with the status “Open”.

table 8 Use Case Descriptions accept or reject offer

Field	Description
Use Case ID	BID-005
Use Case Name	Accept or Reject Offer
Actors	User
Goal	Enable the user to review the offer submitted by the service provider and then accept or reject it.
Preconditions	1. The user is logged in. 2. There is an offer submitted on a request or announcement that belongs to the user.
Main Flow	1. System: Displays the list of offers received on the user’s requests or announcements. 2. User: Reviews a specific offer and clicks “Accept” or “Reject”. 3. System: Updates the status of the selected offer according to the user’s action. 4. User: Returns to the offers list or opens the offer details. 5. System: Displays the updated offer status and shows a success message.
Exceptional Flows	EF1: The offer has already been processed or is no longer available.
Postconditions	The offer status is updated to “Accepted” or “Rejected”.

table 9 Use Case Descriptions submit offer on request

Field	Description
Use Case ID	UC-22
Use Case Name	Submit Offer on Request
Actors	Provider
Goal	Enable the service provider to submit an offer on a service request posted by the user.
Preconditions	1. The service provider is logged in. 2. There is an available service request that can receive offers.
Main Flow	1. System: Displays the list of available service requests. 2. Provider: Selects a request, enters the offer details, and clicks the “Submit Offer” button. 3. System: Validates the entered data and creates a new offer record linked to the service request. 4. Provider: Moves to the provider’s offers list. 5. System: Displays the new offer and shows a success message.

Exceptional Flows	EF1: The request is not available, or the offer data is invalid.
Postconditions	A new offer record is created and linked to the service request.

table 10 Use Case Descriptions add services

Field	Description
Use Case ID	SERV-001
Use Case Name	Add Service
Actors	Provider
Goal	Enable the service provider to add a new service.
Preconditions	1. The service provider is logged in. 2. The Add Service page is available.
Main Flow	1. System: Displays the Add Service form, including name, description, price, and execution duration. 2. Provider: Enters the service details and clicks the “Save” button. 3. System: Validates that all required data is completed and that the price and execution duration are valid. 4. Provider: Confirms the operation. 5. System: Creates a new service record linked to the provider and displays a success message.
Exceptional Flows	EF1: The data is incomplete, or the price/execution duration is invalid.
Postconditions	A new service record is created and linked to the service provider.

table 11 Use Case Descriptions post new request

Field	Description
Use Case ID	UC-21
Use Case Name	Post New Request
Actors	User
Goal	Enable the user to post a service request announcement with a service description and expected budget.
Preconditions	1. The user is logged in. 2. The Post New Request page is available.
Main Flow	1. System: Displays the Post New Request form. 2. User: Enters the service description and expected budget, then clicks the “Post Request” button. 3. System: Validates that the data is complete and that the budget is valid, then creates a new service request record. 4. User: Moves to the user’s requests page or continues browsing. 5. System: Displays the new request in the user’s request list and shows a success message.
Exceptional Flows	EF1: The budget or entered data is invalid.
Postconditions	A new service request record is created in the database.

4.7.3 Activity Diagrams

Activity diagrams describe the workflow of major processes step by step. They are useful for representing decisions, validations, and outcomes in operations that include multiple stages. In this project, the recommended activity diagrams are related to posting a service request, submitting an offer, accepting or rejecting an offer

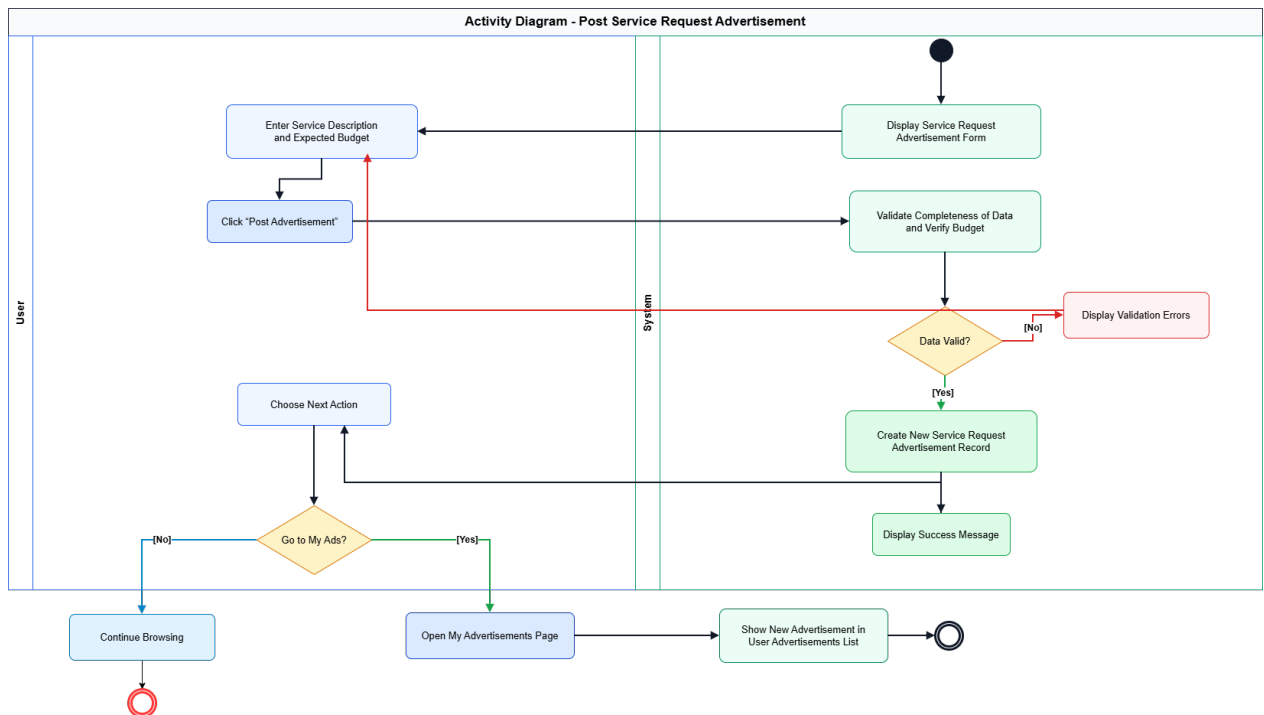


Figure 2 Post Service Request

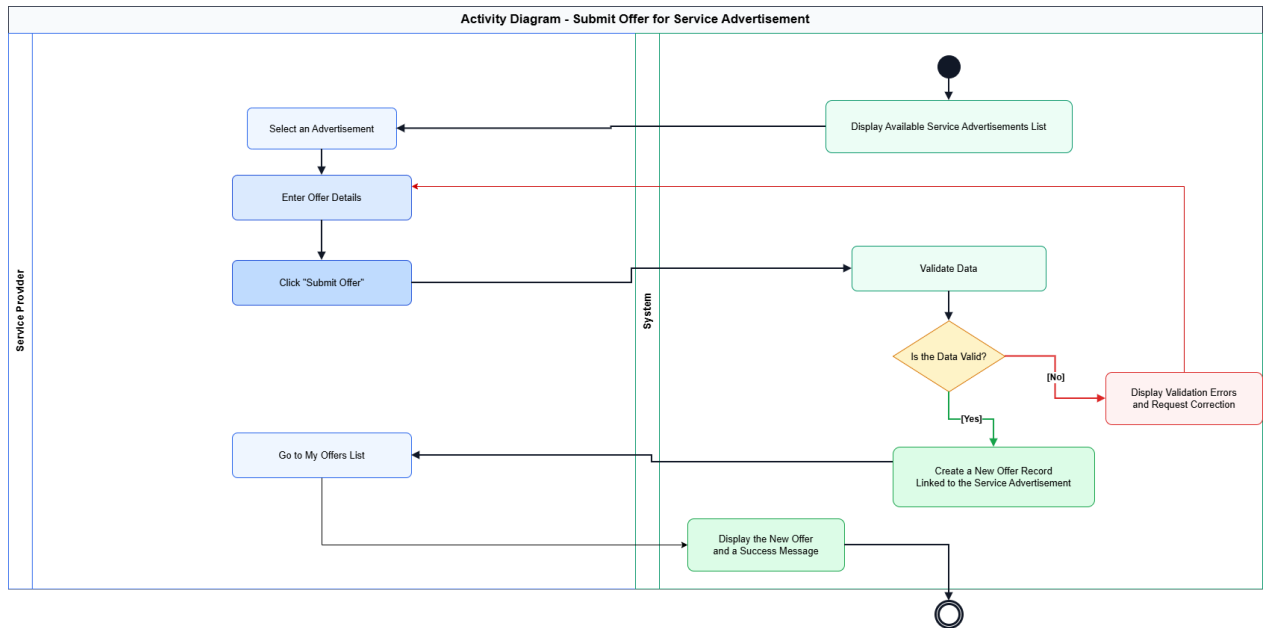


Figure 3 Activity Diagram - Submit Offer

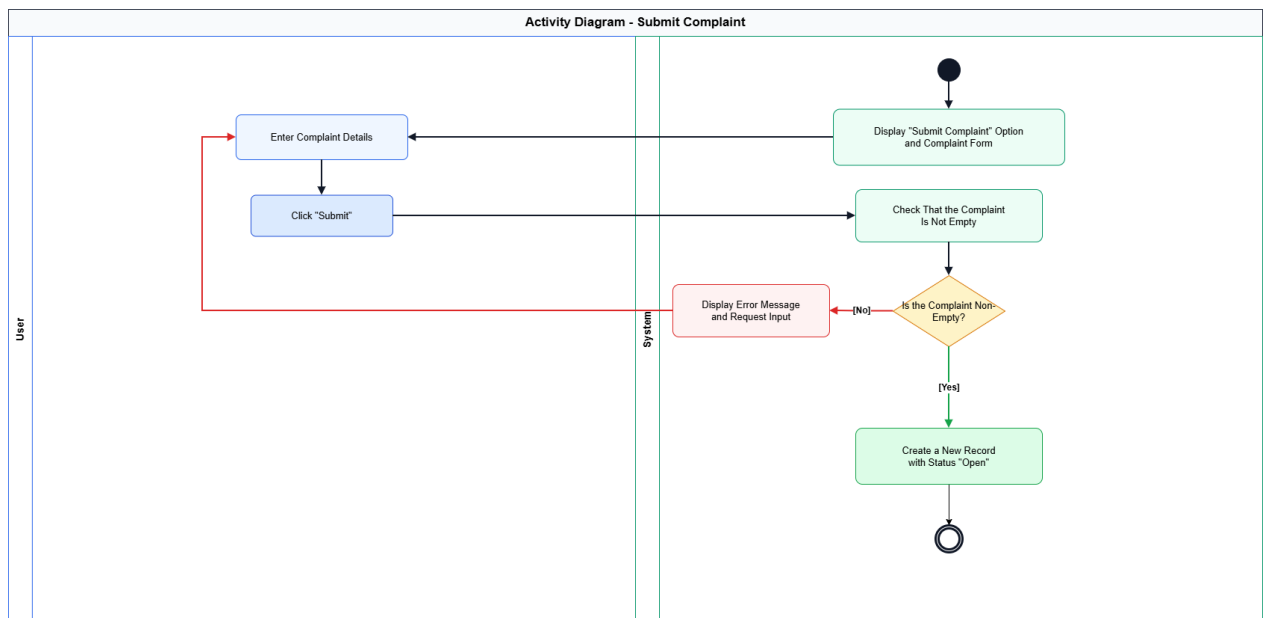


Figure 4 Activity Diagram submit complaint

4.7.4 Sequence Diagrams

Sequence diagrams show the chronological interaction between actors and system components during the execution of major use cases. They are especially useful for clarifying how requests

move between the user interface, the application logic, and the database layer. The following placeholders should be replaced with the final sequence diagrams.

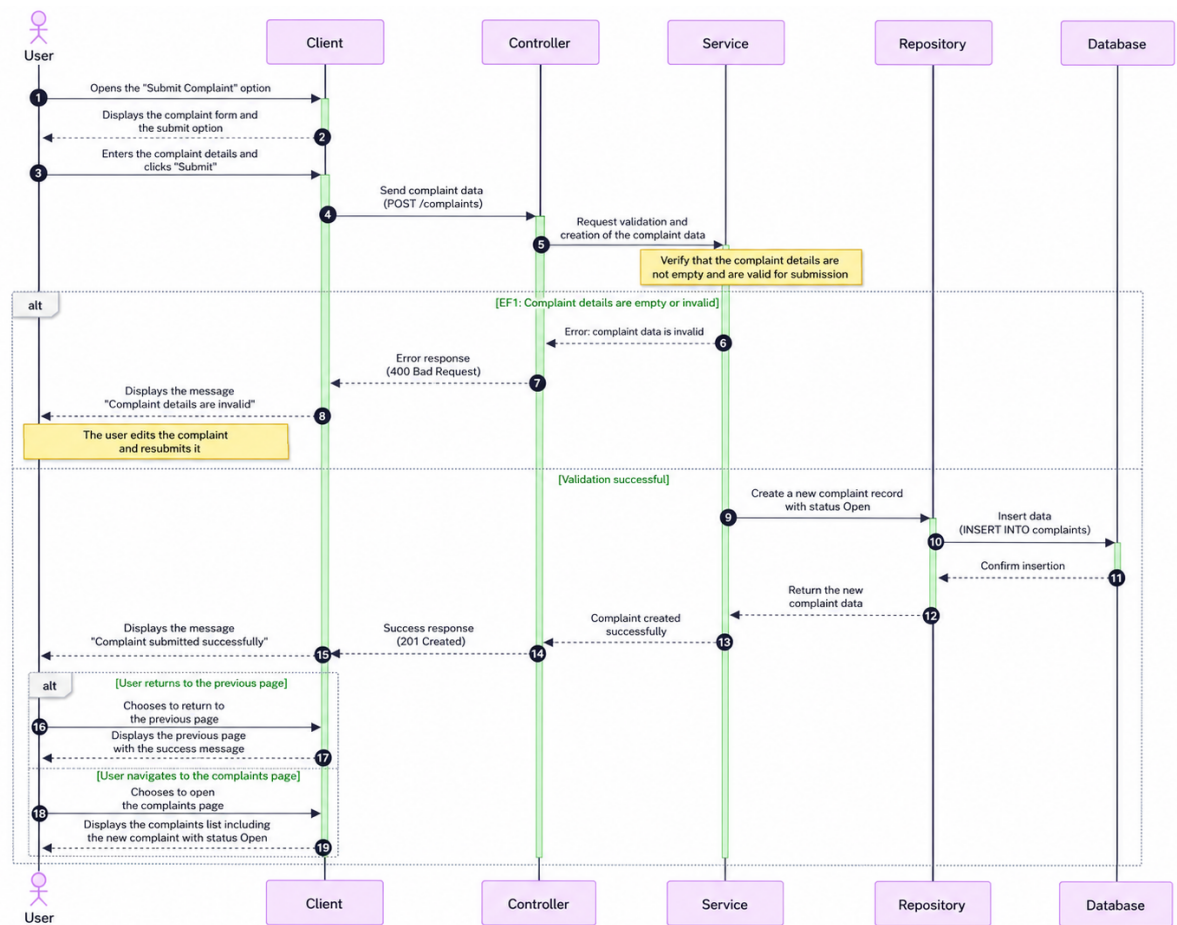


Figure 5 Sequence Diagram - Submit Offer

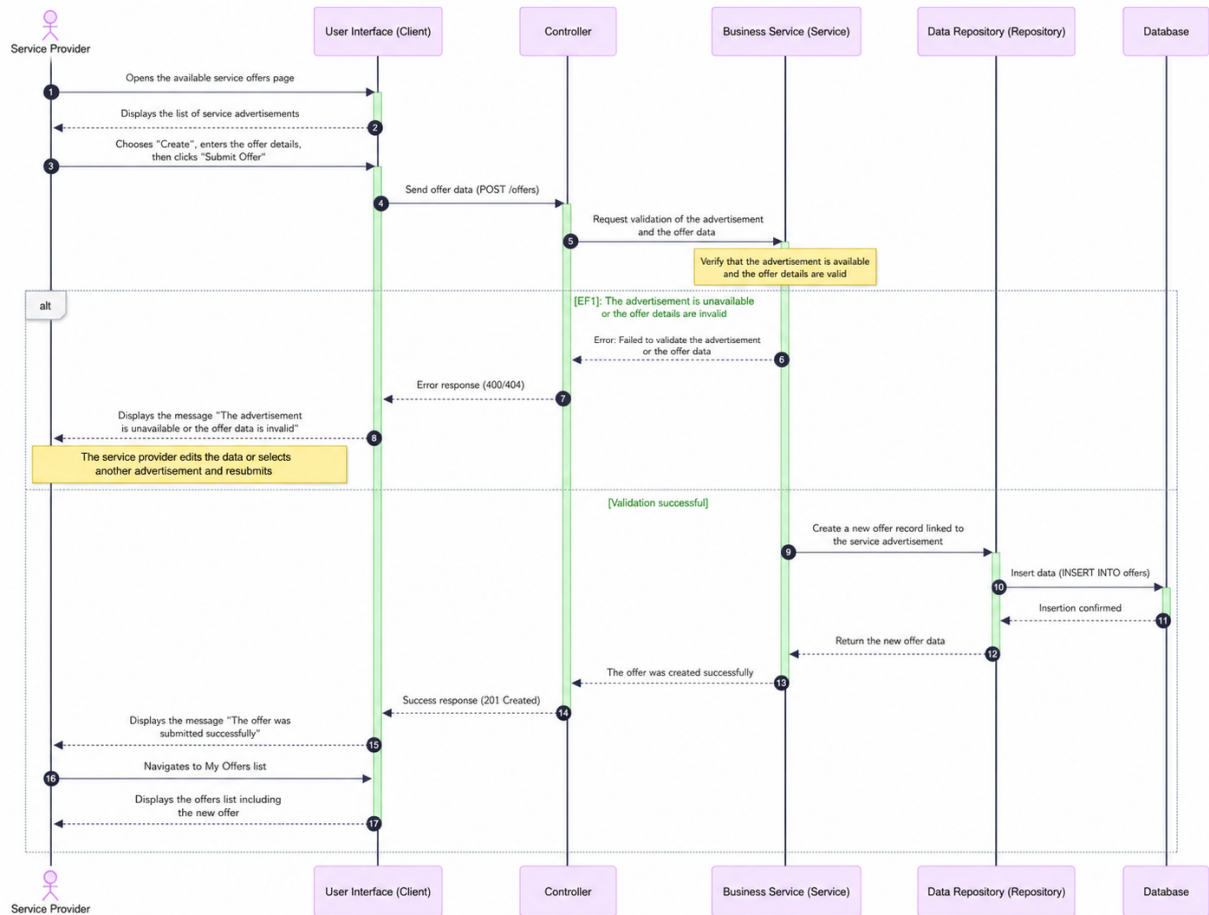


Figure 6 Sequence Diagram - Submit Offer

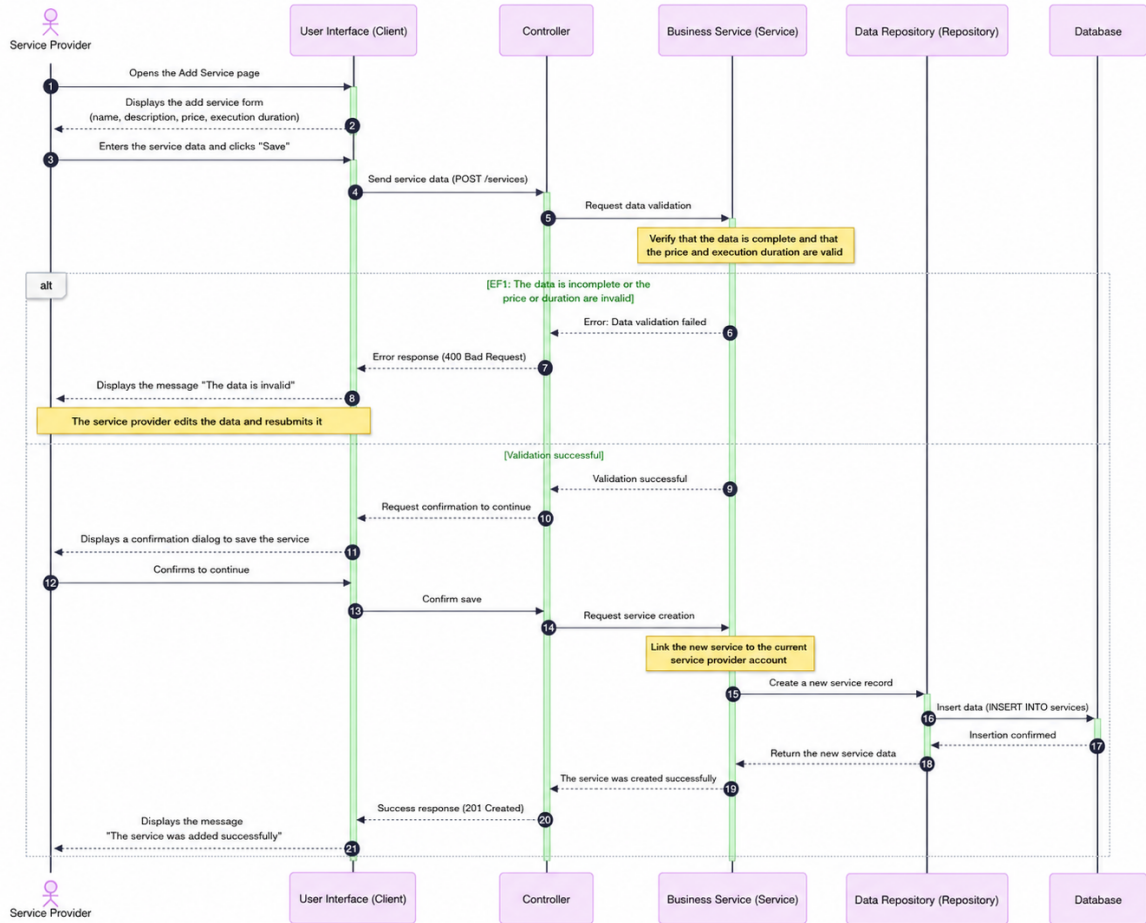


Figure 7 Sequence Diagram - add services

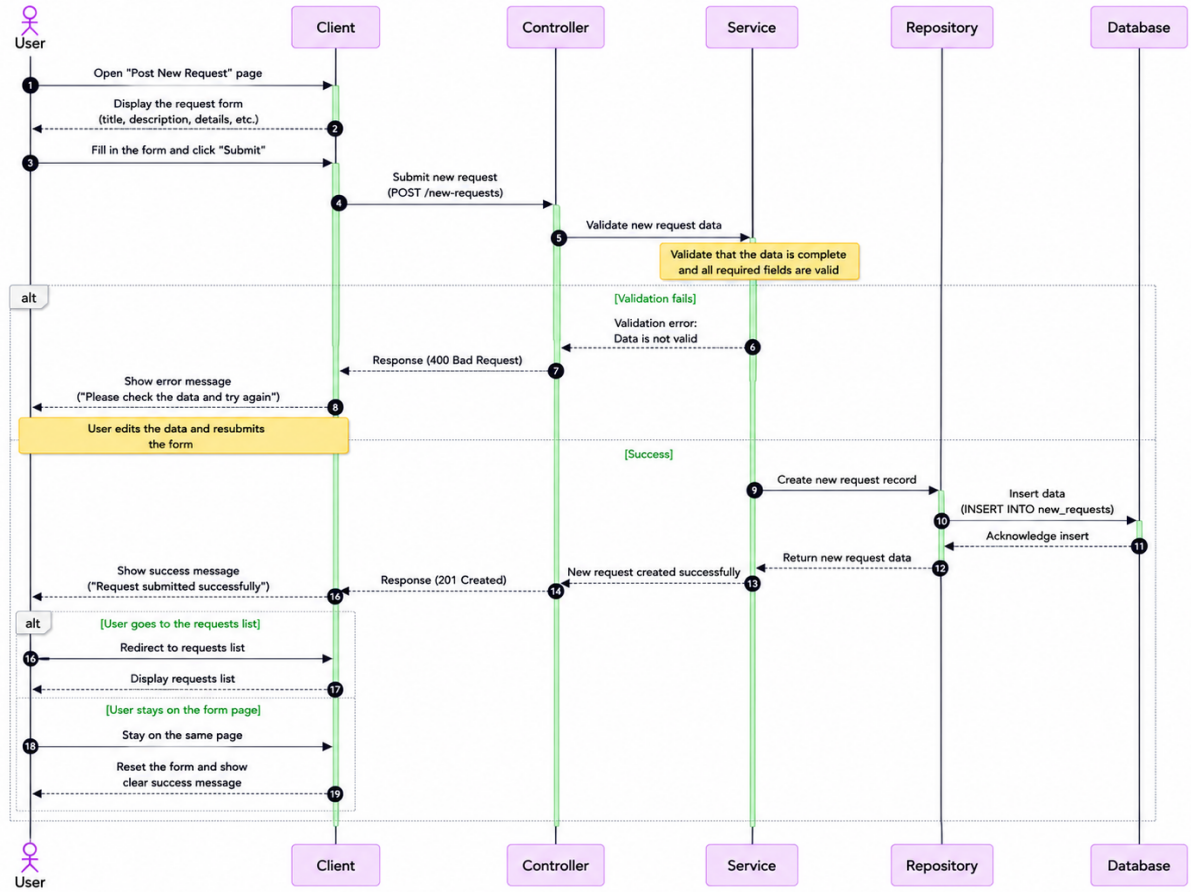


Figure 8 Sequence Diagram -post new request

4.8 Modeling According to the Selected Methodology

Since Syrian Services Hub uses Incremental Development, the requirements are organized into prioritized increments that deliver usable capabilities progressively. The supporting artifacts include user stories, an increment plan, acceptance criteria, dependency awareness, and review outputs for each completed increment. This form of modeling complements UML diagrams by showing how requirements are grouped into coherent deliveries, validated after implementation, and extended in later increments without losing traceability.

4.8.1 Increment Plan and User Stories

table 12 Increment Plan and User Stories

Story ID	User Story	Priority	Acceptance Focus
US-01	As a visitor, I want to browse services so that I can explore the platform before registering.	Must	Services are displayed without requiring login.
US-02	As a user, I want to post a service request so that providers can submit offers.	Must	A request is created and visible to eligible providers.
US-03	As a provider, I want to publish services so that users can request them directly.	Must	A service is created and linked to the provider profile.
US-04	As a provider, I want to submit offers on user requests so that I can compete for work.	Must	An offer is saved and linked to the selected request.
US-05	As a user, I want to compare providers so that I can select the most suitable one.	Should	Provider data is displayed in a comparison view.
US-06	As a user, I want to enter a problem and receive suggested solutions and a provider.	Must	The system displays solution suggestions and a recommended provider.
US-07	As an admin, I want to revoke provider verification so that trust violations can be handled.	Must	The provider verification status changes to unverified and the action is logged.
US-08	As an authenticated user, I want to ask a factual question about a selected provider or service so that I can receive a grounded answer with source citations.	Must	Ask Assistant returns a cited answer from authorized evidence or a clear insufficient-evidence response.
US-09	As a provider, I want to manage customer-facing knowledge sources so that Ask Assistant can answer from current and approved information.	Must	The provider can upload, view status, archive, and reindex owned sources without exposing private content.

4.8.2 Story Mapping

The story map groups the main user activities into meaningful workflow areas. This helps present the project as a set of connected journeys rather than isolated requirements.

table 13 Story Mapping

User Journey Area	Main Activities	Representative Stories
-------------------	-----------------	------------------------

Service Discovery	Browse, search, filter, view service details.	US-01, SERV-004, SERV-005, SERV-006
Service Request Flow	Post request, receive offers, accept or reject offers.	US-02, BID-001, BID-002, BID-005
Provider Workflow	Manage profile, publish services, submit offers, follow offer status.	US-03, US-04, PROV-002, BID-006
Trust and Governance	Ratings, complaints, verification, revoke verification.	REV-001, COMP-001, VER-001, VER-003
Intelligent Support	Enter problem, analyze content, suggest solutions and provider.	US-06, RS-001
Knowledge-Based Assistance	Ask provider-specific questions, retrieve authorized evidence, generate a grounded answer, display citations, and preserve conversation history.	US-08, US-09, RAG-001, RAG-002

4.9 Requirements Analysis and Documentation

Requirements analysis and documentation aim to ensure that every major requirement remains clear, justified, and traceable through later stages of design and testing. The following traceability matrix links representative requirements to their related design artifacts and initial validation methods.

4.9.1 Requirements Traceability Matrix

table 14 Requirements Traceability Matrix

<i>TEST CASES</i>	<i>CODING</i>	<i>DETAILED DESIGN</i>	<i>ANALYSIS</i>	<i>USE CASES</i>	<i>USER STORY</i>	<i>REQ ID (MAPPED FR IDs)</i>
<u>TC-01</u>	<u>IM-01</u>	<u>DE-01</u>	<u>AN-01</u>	<u>UC-01</u>	<u>US-01</u>	SERV-001
<u>TC-02</u>	<u>IM-03</u>	<u>DE-02</u>	<u>AN-02</u>	<u>UC-02</u>	<u>US-02</u>	SERV-008

<u>TC-03</u>	<u>IM-04</u>	<u>DE-03</u>	<u>AN-03</u>	<u>UC-03</u>	<u>US-03</u>	BID-001
<u>TC-04</u>	<u>IM-05</u>	<u>DE-04</u>	<u>AN-04</u>	<u>UC-04</u>	<u>US-04</u>	BID-002
<u>TC-05</u>	<u>IM-05</u>	<u>DE-05</u>	<u>AN-05</u>	<u>UC-05</u>	<u>US-05</u>	AUTH-001
TC-08	IM-08	DE-08	<u>AN-08</u>	<u>UC-08</u>	<u>US-08</u> / <u>US-09</u>	RAG-001 / RAG-002

4.9.2 Initial Test Cases

Initial test cases provide an early bridge between requirement analysis and later verification. They do not replace the full testing chapter, but they show how the most important requirements can be translated into observable system behavior.

table 15 Initial Test Cases-add services

Test Case ID	TC-SERV-001	Requirement Ref.	SERV-001
Use Case Ref.	UC-10	Module	Services
Title	Add Service	Owner / Actor	Provider
Test Type	Functional / Positive	Test Level	System
Priority	High	Severity	High
Risk Level	High	Automation Candidate	Medium
Objective	Verify that the system satisfies the Add Service requirement according to the defined main scenario.	Design Technique	Use Case Based
Description		Allows the service provider to add a new service that includes service name, description, price, and execution duration.	

Pre-conditions		The required actor is logged in. The Add Service page is available.	
Test Data / Input		Service Name: AC Maintenance Price: 150 Execution Duration: Two days	
Test Steps		Open the screen or section related to the "Add Service" feature. Enter or select valid test data when needed. Execute the main action related to the feature. Review the result displayed to the user or actor. Verify that the change is saved or data is retrieved from the database according to the expected behavior.	
Expected Result		A new service record linked to the service provider is created. No unexpected error messages or behavior that conflicts with the scenario appear. The interface or data is updated according to the performed operation.	
Post-conditions		A new service record linked to the service provider is created.	
Exception Notes		There are no additional exception notes for this baseline case; other scenarios are covered in later test cases.	
Actual Result		Execution Status	☐ Pass ☐ Fail ☐ Blocked ☐ N/A
Defect Reference		Executed By	
Reviewed By		Date Created	2026-03-10
Execution Date		Remarks / Notes	These fields are used during actual execution by the testing team.

table 16 Initial Test Cases send offer

Test Case ID	TC-SERV-008	Requirement Ref.	SERV-008
Use Case Ref.	UC-17	Module	Orders
Title	Send Offer on Service Request	Owner / Actor	Provider
Test Type	Functional / Positive	Test Level	System
Priority	High	Severity	High
Risk Level	High	Automation Candidate	Medium
Objective	Verify that the system satisfies the Send Offer on Service Request requirement according to the defined main scenario.	Design Technique	Use Case Based
Description	Allows the service provider to respond to the service request sent by the user by submitting an offer that includes price, duration, or execution details.		
Pre-conditions	The required actor is logged in. An incoming service request with Pending status exists.		
Test Data / Input	Proposed Price: 120 Proposed Duration: 2 days Execution Details: Visit and inspection		
Test Steps	Open the screen or section related to the "Send Offer on Service Request" feature. Enter or select valid test data when needed. Execute the main action related to the feature. Review the result displayed to the user or actor. Verify that the change is saved or data is retrieved from the database according to the expected behavior.		
Expected Result	A new offer record linked to the service request is created, and the request status is updated to indicate that it has been answered with an offer. No unexpected error messages or behavior that conflicts with the scenario appear. The interface or data is updated according to the performed operation. In the exceptional scenario: the offer data is invalid or the request is no longer Pending.		

Post-conditions		A new offer record linked to the service request is created, and the request status is updated to indicate that it has been answered with an offer.	
Exception Notes		The offer data is invalid or the request is no longer Pending.	
Actual Result		Execution Status	☐ Pass ☐ Fail ☐ Blocked ☐ N/A
Defect Reference		Executed By	
Reviewed By		Date Created	2026-03-10
Execution Date		Remarks / Notes	These fields are used during actual execution by the testing team.

table 17 Initial Test Cases post new request

Test Case ID	TC-BID-001	Requirement Ref.	BID-001
Use Case Ref.	UC-21	Module	Bidding
Title	Post New Request	Owner / Actor	User
Test Type	Functional / Positive	Test Level	System
Priority	High	Severity	High
Risk Level	High	Automation Candidate	Medium
Objective	Verify that the system satisfies the Post New Request requirement according to the defined main scenario.	Design Technique	Use Case Based
Description		Allows the user to post a service request with a service description and expected budget.	
Pre-conditions		The required actor is logged in. The Post New Request page is available.	
Test Data / Input		Description: Installing an electrical network Expected Budget: 500	
Test Steps		Open the screen or section related to the "Post New Request" feature.	

		Enter or select valid test data when needed. Execute the main action related to the feature. Review the result displayed to the user or actor. Verify that the change is saved or data is retrieved from the database according to the expected behavior.	
Expected Result		A new service request record is created in the database. No unexpected error messages or behavior that conflicts with the scenario appear. The interface or data is updated according to the performed operation.	
Post-conditions		A new service request record is created in the database.	
Exception Notes		There are no additional exception notes for this baseline case; other scenarios are covered in later test cases.	
Actual Result		Execution Status	☐ Pass ☐ Fail ☐ Blocked ☐ N/A
Defect Reference		Executed By	
Reviewed By		Date Created	2026-03-10
Execution Date		Remarks / Notes	These fields are used during actual execution by the testing team.

Section Three: Design and Architecture

Chapter Five: System Architecture and Design

This chapter explains how the analyzed requirements of Syrian Services Hub are transformed into a structured technical design. It presents the architectural vision of the platform, the logic behind its main components, and the way system functionality is distributed across the application structure. The chapter also covers database design, API-oriented interaction, and security considerations. In this way, the chapter serves as the bridge between requirements analysis and implementation by showing how the system is planned to operate from a software engineering perspective.

5.1 High-Level System Design

The high-level design of Syrian Services Hub focuses on organizing the platform into clear functional layers and modules that support service discovery, service-demand advertisements, offer management, communication, trust-related operations, and recommendation support. At this level, the design aims to ensure maintainability, separation of concerns, and a coherent interaction between the client interface, the backend application logic, and the underlying data storage.

5.1.1 System Block Diagram

The system block diagram presents the main components of the platform and the communication paths between them. It should show the user-facing client side, the server-side application logic, the database layer, and the recommendation-related module. The figure is intended to summarize the overall structure of the platform before moving into more detailed architectural explanation.

SYRIAN SERVICES HUB - SYSTEM BLOCK DIAGRAM

High-Level Architecture (Client-Server, 3-Layer inside Modular Monolith)

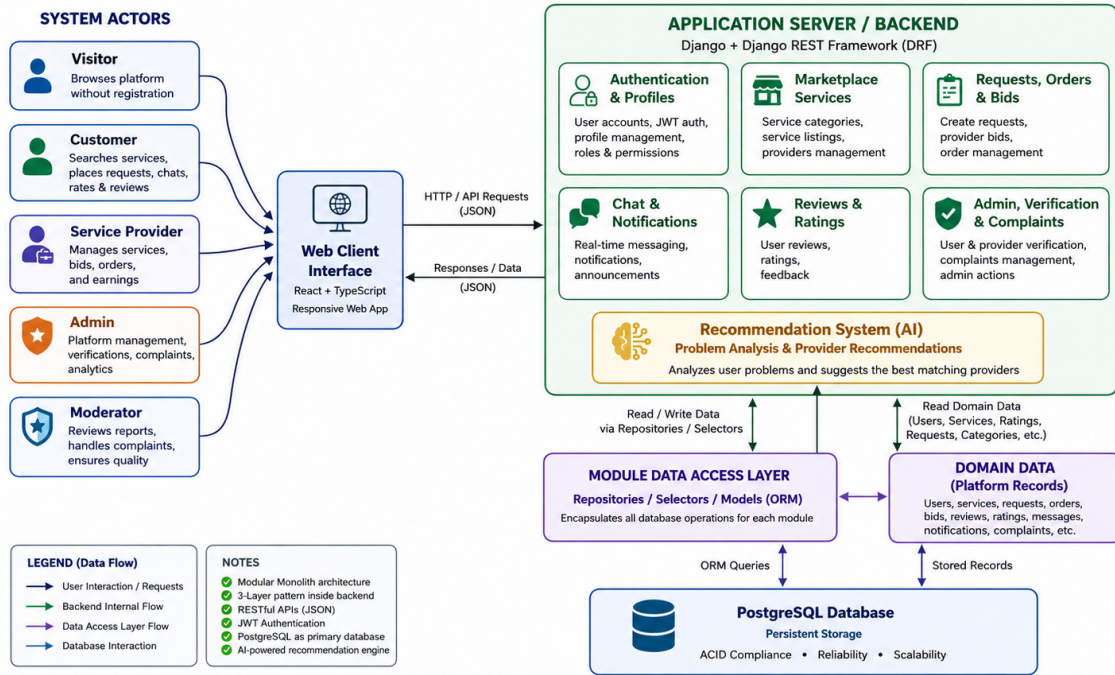


Figure 9 System Block Diagram of Syrian Services Hub

5.1.2 Architectural Pattern Used

Syrian Services Hub follows a client-server architecture implemented through a three-tier design pattern. This architectural pattern was selected because it provides clear separation between the presentation layer, the application or business logic layer, and the data layer. Such separation is well suited to a web-based platform that must support multiple user roles, structured service workflows, and recommendation-oriented processing while remaining maintainable and scalable.

table 20 Architecture Layer Summary

Layer	Main Responsibility	Technology Direction
Presentation Layer	Provides the user interface for visitors, users, providers, and administrators; supports browsing, requests, offers, dashboards, and recommendation interaction.	React-based web interface
Business Logic Layer	Handles workflows, validation, role-based operations, recommendation	Django application logic and REST endpoints

	processing, notifications, complaints, and verification management.	
Data Layer	Stores persistent platform records such as users, services, requests, offers, messages, complaints, ratings, and verification data.	PostgreSQL relational database

5.1.3 System Architecture Design

Syrian Services Hub follows a **three-layer architecture**: presentation, business logic, and data. The presentation layer manages role-based interfaces, the business logic layer handles workflows, authorization, and intelligent processing, and the data layer stores persistent relational data.

AI Recommendation analyzes user needs to suggest suitable services and providers, while **Ask Assistant** uses **RAG** to answer provider-specific questions from authorized knowledge sources.

5.1.4 Ask Assistant RAG Architecture

Ask Assistant operates through the **knowledge** and **customer_assistant** modules. The knowledge module manages sources, chunks, embeddings, and retrieval, while customer_assistant manages sessions, questions, answers, and citations.

Authorized documents are validated, chunked, embedded, and indexed. During retrieval, access controls are applied before relevant evidence is selected for answer generation. The system follows a **grounding-first security model**, excluding unauthorized content and returning an insufficient-evidence response when reliable information is unavailable.

5.2 Database Design

The system uses a **relational database model** to maintain consistency across users, providers, services, requests, offers, messages, ratings, complaints, verification records, and administrative activities.

5.2.1 Entity Relationship Model

The Entity Relationship Model defines the relationships among the platform's main entities, including **users**, **provider profiles**, **services**, **requests**, **offers**, **conversations**, **messages**, **ratings**, **complaints**, and **verification requests**, supporting both direct services and request-and-offer workflows.

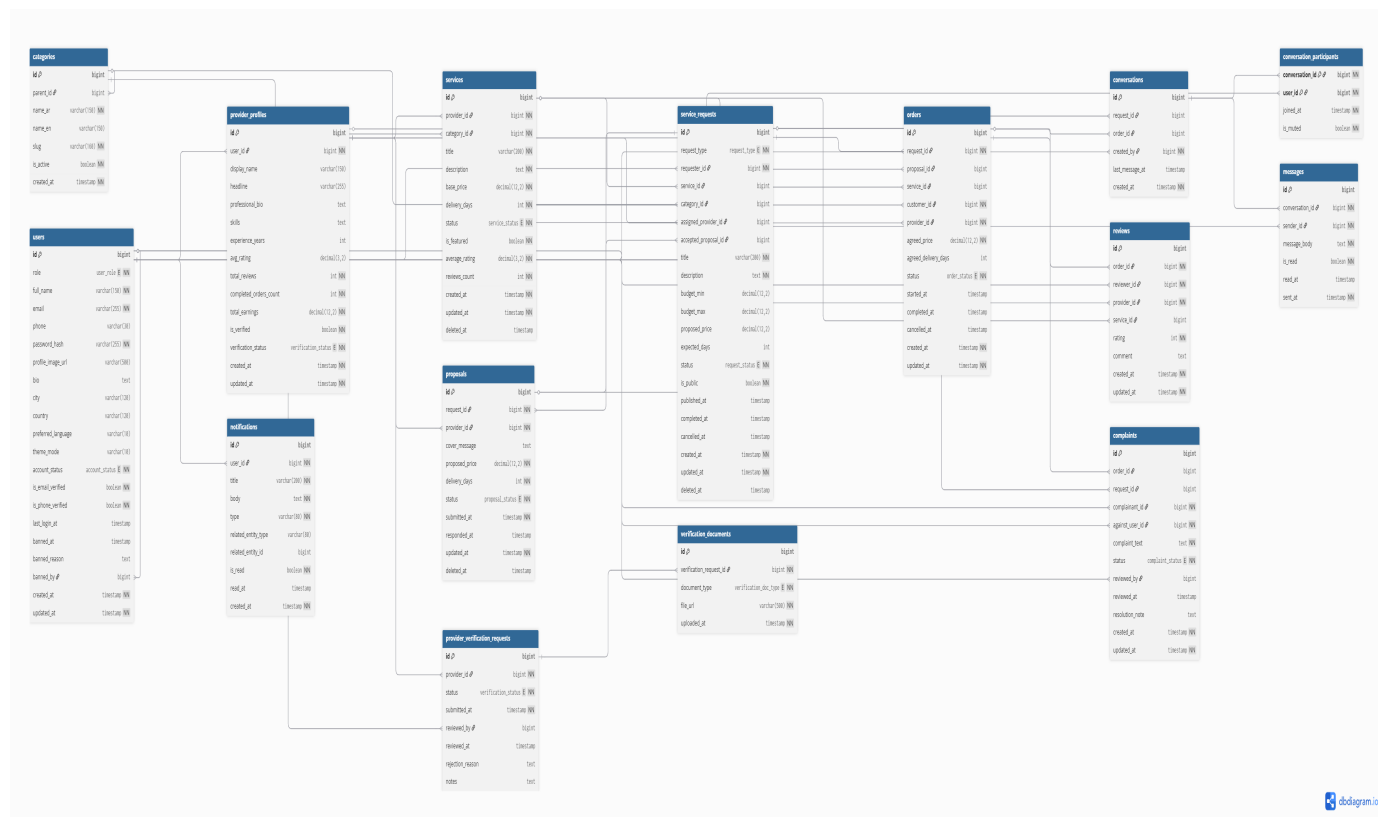


Figure 10 ER Diagram of Syrian Services Hub

5.2.2 Relational Schema

The relational schema translates the conceptual entity model into database tables with identifiable keys and structured attributes. Table 5.2 summarizes the main tables required to support the platform logic.

table 21 Core Relational Schema of Syrian Services Hub

Table Name	Primary Key	Selected Attributes	Purpose
------------	-------------	---------------------	---------

User	user_id	full_name,email, password_hash, role, phone, is_active	Stores platform accounts for visitors after registration, users, providers, and administrators.
ProviderProfile	provider_profile_id	user_id, bio, location, skills, verification_status	Stores professional information related to a service provider.
Category	category_id	name, description	Organizes services into searchable categories.
Service	service_id	provider_profile_id, category_id, title, description, price, delivery_time, status	Stores ready-made services published by providers.
ServiceRequest	request_id	user_id, category_id, title, problem_description, budget, status, created_at	Stores open service-demand advertisements posted by users.
Offer	offer_id	request_id, provider_profile_id, offer_price, delivery_time, proposal_text, status	Stores provider offers submitted in response to service requests.
Conversation	conversation_id	user_id, provider_profile_id, request_id, created_at	Represents a communication channel between the involved parties.
Message	message_id	conversation_id, sender_user_id, body, sent_at, read_status	Stores messages exchanged within a conversation.
Rating	rating_id	service_id, rater_user_id, provider_profile_id, score, feedback, created_at	Stores ratings and feedback after service completion.
Complaint	complaint_id	submitted_by, target_user_id, reason, details, status, created_at	Stores platform complaints and moderation records.
VerificationRequest	verification_id	provider_profile_id, submitted_at, status, reviewed_by, reviewed_at	Stores provider verification submissions and review outcomes.

KnowledgeSource	knowledge_source_id	provider_id, source_type, title, visibility, status, checksum_sha256, created_at, updated_a	Stores provider-owned customer-facing sources and their ingestion state.
KnowledgeChunk	knowledge_chunk_id	document_id, source_id, provider_id, chunk_index, chunk_text, embedding, token_count, visibility, metadata, is_active	Stores searchable evidence segments and vector embeddings used by Ask Assistant.
AssistantSession	assistant_session_id	customer_id, provider_id, order_id, service_id, status, created_at, updated_at	Groups the customer conversation within a provider-specific context.
AssistantTurn	assistant_turn_id	session_id, customer_id, provider_id, question, answer, answer_status, model, input_tokens, output_tokens, latency_ms, created_at	Stores customer questions and assistant answers in chronological order.
AssistantCitation	assistant_citation_id	turn_id, chunk_id, source_id, provider_id, quote, relevance_score, rank, created_a	Links a generated answer to the exact supporting evidence displayed to the user.

In addition to the core tables above, the final implementation may also include supporting tables such as Notification, Attachment, or AuditLog depending on the chosen level of administrative control and traceability.

5.2.3 Constraints and Data Integrity Rules

To preserve consistency and prevent invalid platform states, the relational schema uses primary keys, foreign keys, uniqueness rules, and selected validation constraints. Table 5.3 summarizes representative constraints applied to major entities.

table 22 Main Database Constraints

Table	Constraint Type	Rule	Purpose
User	UNIQUE	email must be unique	Prevents duplicate accounts with the same email.

User	CHECK	role IN (visitor, user, provider, admin)	Restricts the role to valid system roles.
ProviderProfile	FOREIGN KEY	user_id references User(user_id)	Ensures every provider profile belongs to a valid user account.
Service	FOREIGN KEY	provider_profile_id references ProviderProfile(provider_profile_id)	Prevents orphan service records.
Service	CHECK	price >= 0 and delivery_time > 0	Prevents invalid service values.
ServiceRequest	FOREIGN KEY	user_id references User(user_id)	Links each request to its owner.
Offer	FOREIGN KEY	request_id references ServiceRequest(request_id)	Ensures every offer belongs to a valid request.
Message	FOREIGN KEY	conversation_id references Conversation(conversation_id)	Preserves message-conversation integrity.
VerificationRequest	CHECK	status IN (pending, approved, rejected, revoked)	Supports both granting and revoking verification status.
KnowledgeSource	OWNERSHIP STATUS /	provider_id must match the selected provider; only active customer-visible sources may be retrieved	Prevents cross-provider and inactive-source leakage.
KnowledgeChunk	VECTOR / FK	embedding is stored with a valid source reference and chunk order	Supports similarity retrieval while preserving traceability to the source.
AssistantSession	OWNERSHIP	customer_id and provider_id define the authorized conversation scope	Prevents a session from searching unrelated providers.
AssistantCitation	FK / INTEGRITY	each citation must reference the assistant turn and a retrieved chunk from an authorized source	Ensures every displayed citation can be traced to evidence.

5.3 External API Design

Syrian Services Hub is designed as a web platform in which the frontend interacts with the backend through REST-oriented endpoints. These endpoints allow the client side to retrieve

services, submit requests, manage offers, exchange messages, and perform trust-related operations in a structured and predictable way. The API layer also allows the recommendation feature to be requested from the user interface in a standardized form.

Table 5.4 presents representative endpoints used to support the main platform workflows. The final implementation may contain additional endpoints depending on modular refinement and administrative needs.

table 23 Representative API Endpoints

Method	Endpoint	Main Function	Role	Response Type
POST	/api/auth/register	Register account	User / Provider	JSON
POST	/api/auth/login	User login	All registered roles	JSON
GET	/api/services	Browse services	Visitor / User	JSON list
POST	/api/services	Create service	Provider	JSON
POST	/api/requests	Create service request	User	JSON
POST	/api/offers	Submit offer	Provider	JSON
GET	/api/conversations/{id}/messages	Get messages	Authorized parties	JSON list
POST	/api/recommendations	Get suggested solutions and provider	User	JSON
POST	/api/verification-requests	Submit verification request	Provider	JSON
PATCH	/api/admin/providers/{id}/revoke-verification	Revoke verification	Admin	JSON
POST	/api/v1/customer-assistant/questions/	Ask a provider-specific question and receive a grounded cited answer	Authenticated User	JSON
GET	/api/v1/customer-assistant/sessions/{id}/turns/	Retrieve authorized Ask Assistant	Authenticated User	JSON

		conversation history		
POST	/api/v1/knowledge/sources/	Upload a provider-owned customer-facing knowledge source	Provider	Multipart/form-data
POST	/api/v1/knowledge/sources/{id}/reindex/	Reprocess and re-embed an owned knowledge source	Provider / Admin	JSON

5.4 Security and Access Design

Security in Syrian Services Hub is based on **authentication, role-based access control, protected workflows, and administrative oversight**. Sensitive operations are restricted according to user roles, while trust-related features such as complaints and verification are managed through controlled processes.

Ask Assistant applies additional safeguards through **permission-first retrieval**, source visibility checks, citation validation, file validation, rate limiting, audit metadata, and insufficient-evidence handling. Retrieved content is treated as untrusted data to reduce prompt-injection, information leakage, and unsupported responses.

5.4.1 Authentication and Authorization

Registered operations require authenticated access, while permissions are assigned according to the **user, provider, or administrator** role. Providers can publish services, users can submit service requests, and administrators manage verification and other privileged actions.

5.4.2 Verification and Revocation Control

Provider verification is an administrative process in which submitted evidence is reviewed and either approved or rejected. Administrators may also revoke verification when necessary, ensuring that trust status remains controlled and valid over time

Section Four:

Implementation and

Testing

Chapter Six: Implementation Environment and Tools

This chapter documents the practical environment used to build Syrian Services Hub and explains the main tools, frameworks, libraries, and execution settings that supported the implementation process. It describes the software stack used for frontend and backend development, the database platform, the testing utilities, dependency management, and the source-control practices applied during development. The chapter also clarifies the relationship between the selected technologies and the system architecture, especially in supporting the client-server and three-tier design adopted by the project.

6.1 Software Tools and Libraries Used

The implementation of Syrian Services Hub relied on a modern web-development stack supported by practical development and testing tools. The selected tools were chosen to match the requirements of a role-based service marketplace platform with structured workflows, database persistence, API communication, and recommendation-oriented processing. The toolset combines frontend development, backend implementation, database management, source control, and API inspection in a way that supports maintainability and modular growth.

table 24 Main software tools and libraries used in the implementation environment

Tool / Library	Role in the Project	Usage Context
Visual Studio Code	Primary development environment	Used for editing source files, project navigation, debugging, and managing the implementation workflow.
React	Frontend user interface library	Used to build the client-side pages, reusable components, dashboards, and interaction flows.
TypeScript	Frontend logic and interactivity	Used to implement component behavior, form interaction, API calls, and client-side state handling.
Django	Backend web framework	Used to implement business logic, role-based workflows, request handling, recommendation processing, and server-side validation.
PostgreSQL	Database management system	Used to store users, provider profiles, services, requests, offers, messages, verification records, and other persistent data.
Postman	API testing and inspection tool	Used to test endpoints, verify request-response behavior, and inspect backend API functionality.

Git / GitHub	Source control and revision tracking	Used to manage versions of the codebase and maintain development history.
OpenAI Python SDK	Embedding and answer-generation integration	Used by the RAG services to create vector embeddings and generate grounded answers from retrieved evidence.
pypdf, python-docx, openpyxl, tiktoken	Document extraction and token-aware chunking	Supports ingestion of approved provider files and preparation of searchable knowledge chunks.

6.2 Development Environment and Project Structure

Syrian Services Hub follows a structured development environment based on the adopted **three-tier architecture**, separating the frontend, backend, and database layers. This organization improves modularity and keeps interface components, business logic, and persistent data clearly separated.

The frontend and backend operate as coordinated application layers, while the database manages persistent records. This structure supports incremental development and testing of services, requests, offers, messaging, trust-related features, and recommendation functionality while preserving clear architectural boundaries.

table 25 High-level implementation environment structure

Project Layer	Representative Elements	Purpose
Client Layer	React pages, components, forms, dashboards	Provides the user-facing interface for visitors, users, providers, and administrators.
Application Layer	Django views, serializers, business logic, recommendation processing	Handles validation, workflows, role-based actions, and system behavior.
Data Layer	PostgreSQL tables, relations, stored records	Maintains persistent data and supports retrieval, updates, and structured relationships.

6.3 Programming Languages and Frameworks

The implementation of the platform combined frontend and backend technologies that are widely used in modern web systems. React and TypeScript were used on the client side to support dynamic interaction, reusable interface components, stronger type safety, and

responsive user flows. On the server side, Django provided the backend framework responsible for routing, validation, workflow handling, recommendation-related processing, and data interaction. This combination was appropriate for a marketplace platform that requires clear separation between user interaction and backend business logic while still maintaining practical development efficiency.

table 26 Programming languages and frameworks used by the project

Technology	Type	Contribution to the System
TypeScript	Programming language	Implements client-side interactivity, request handling, and frontend behavior.
React	Frontend framework/library	Builds dynamic user interfaces and reusable visual components.
Python	Programming language	Supports backend implementation through the Django framework.
Django	Backend framework	Implements APIs, workflow logic, data validation, and recommendation-related processing.
SQL	Database query language	Supports data definition and data manipulation through the PostgreSQL backend.
OpenAI API	AI service integration	Provides embedding generation and evidence-constrained response generation for Ask Assistant.

6.4 Database Server and Operating System

Syrian Services Hub uses **PostgreSQL** as its primary database server due to its reliability, relational data support, and compatibility with modern backend frameworks. It manages interconnected entities such as users, services, requests, offers, messages, complaints, verification records, and recommendation-related data.

Development is carried out in a standard local operating-system environment capable of running and integrating the frontend, backend, and database components without requiring specialized infrastructure during the development stage.

table 27 Database and execution environment summary

Environment Element	Selected Option	Reason for Selection
---------------------	-----------------	----------------------

Database Server	PostgreSQL	Provides reliable relational persistence, strong integrity support, and suitability for multi-entity platform data.
Execution Model	Local development environment	Supports iterative testing and controlled feature verification during implementation.
System Context	Web-based client-server environment	Matches the platform architecture and the requirement for role-based online interaction.

6.5 Dependencies

The project depends on a set of framework and runtime packages required to support frontend rendering, backend execution, database connectivity, and API communication. In the backend environment, dependencies are managed through Python package installation, while the frontend relies on a TypeScript-based toolchain and package management for component rendering and interface behavior. Dependency tracking is important because it ensures that the platform can be rebuilt consistently and that the implementation environment remains reproducible.

table 28 Main dependency groups used by the project

Dependency Group	Examples	Function
Backend packages	Django and related Python packages	Provide routing, validation, ORM support, and server-side functionality.
Frontend packages	React and TypeScript dependencies	Provide rendering, state management, and component-based interface behavior.
Database connector packages	PostgreSQL integration packages	Enable communication between backend logic and persistent storage.
Testing / utility packages	Request inspection and development utilities	Support API validation, debugging, and controlled implementation checks.
RAG and document-processing packages	openai,,pypdf,,python-docx, openpyxl,	Provide embeddings, grounded answer generation, file extraction,

6.6 Version Control and Source Management Tools

Version control was handled through Git and GitHub in order to preserve the development history of Syrian Services Hub and reduce the risk of losing implementation progress during iterative work. Source control is especially important in a project that evolves across analysis,

design, and implementation because code revisions must remain traceable and recoverable. Git-based source management also supports structured modification of frontend and backend modules, incremental refinement of project files, and safer maintenance of the codebase over time.

table 29 Version control and source-management tools

Tool	Purpose	Practical Benefit
Git	Version control	Tracks changes, supports rollback, and preserves implementation history.
GitHub	Repository hosting and collaboration	Stores the codebase remotely and supports organized source management.

6.7 Continuous Integration and Deployment Tools

At the current project stage, Syrian Services Hub did not rely on a dedicated automated continuous-integration or deployment pipeline. The implementation process focused on controlled local development, iterative testing, and direct verification of frontend-backend interaction. This is acceptable within the scope of an academic graduation project because the primary objective at this stage is to build, validate, and document the core system rather than establish a production-grade deployment workflow. Nevertheless, the project structure remains suitable for future extension toward CI/CD integration if the system is later developed into a larger deployable platform.

table 30 CI/CD status within the current project scope

Aspect	Current Status	Future Possibility
Continuous Integration	Not automated in the current academic scope	Can be extended later through automated testing and build pipelines.
Continuous Deployment	Not applied at the current project stage	Can be introduced when the system moves toward hosted production deployment.

6.8 Testing Environment

Syrian Services Hub was tested primarily in a **local development environment** integrating the React frontend, Django backend, and PostgreSQL database. **Postman** was also used to validate API behavior independently when required.

Testing focused on role-based access, service workflows, request and offer handling, messaging, verification, recommendation responses, and data persistence. This setup supported rapid debugging, repeated validation, and efficient incremental testing throughout development.

table 31 Testing environment elements used during implementation

Testing Element	Environment / Tool	Purpose
Frontend validation	Local browser execution	Checks interface behavior, navigation, forms, and dashboard interaction.
Backend validation	Local Django execution	Checks logic processing, endpoint behavior, and workflow correctness.
API testing	Postman	Validates request-response structures and endpoint correctness.
Database validation	PostgreSQL local execution	Checks data persistence, relations, and stored results after operations.
RAG validation	Django/DRF test client, pytest, PostgreSQL, mocked or controlled OpenAI responses	Checks ingestion, permission filtering, retrieval relevance, citations, insufficient-evidence behavior, and API contracts.

Overall, the implementation environment of Syrian Services Hub reflects a practical and coherent web-development setup that is consistent with the system architecture, the project requirements, and the academic scope of the graduation project.

Chapter Seven: Software Implementation

This chapter presents the practical implementation of Syrian Services Hub and explains how the designed architecture was translated into an operational software structure. It covers the project folder organization, the implementation of the main functional modules, the API layer, the data-access responsibilities, the business-logic layer, the user interface, and the implementation status of the intelligent recommendation-oriented functionality.

7.1 Project Structure and File Organization

The implementation of Syrian Services Hub follows a repository structure that separates the frontend, backend, documentation, and tracking assets of the project. According to the repository definition, the platform is built as a full-stack service marketplace with a React and TypeScript frontend and a Django plus Django REST Framework backend. The codebase is organized to support incremental development, clear domain separation, and maintainable delivery artifacts.

table 32 High-level repository structure of Syrian Services Hub

Directory	Purpose
frontend/	Contains the client-side application, user interface pages, routing, state management, and API integration logic.
backend/	Contains the Django and DRF server-side code, domain modules, authentication, business rules, and API endpoints.
docs/	Contains architecture, system, API, security, and delivery documentation.
tracking/	Contains work-management records, decisions, risks, changelog notes, and open implementation questions.
.vscode/ and root files	Contain local editor configuration and repository-level setup files.

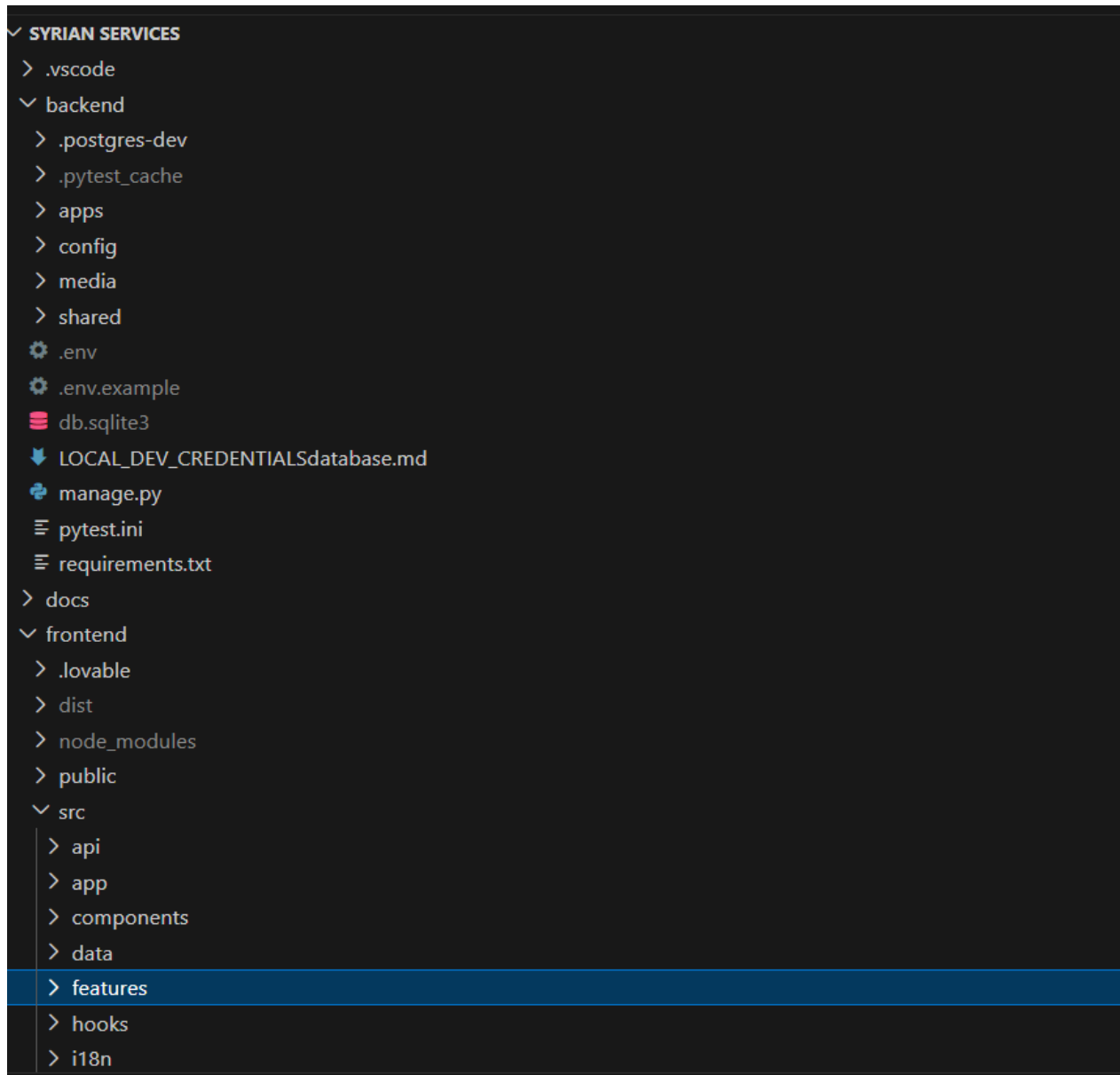


Figure 11 Placeholder for the project structure

7.2 Implementation of Major Functional Modules

The backend implementation is organized around a set of domain-oriented modules that collectively realize the service marketplace workflows of the platform. The repository identifies the core domains as accounts, providers, services, orders, bids, chat, complaints, reviews, notifications, and admin_panel. In practical terms, these modules implement the functional scope discussed in the requirements chapter, including user accounts, provider profiles, service publishing, service-demand workflows, offer submission, communication, trust-related actions,

and administrative supervision. This modular organization improves maintainability by separating concerns while keeping all business capabilities within a unified backend project.

table 33 Main implementation modules and their responsibilities

Module	Main Responsibility
accounts	User registration, login, session handling, profile basics, and role-aware account operations.
providers	Provider-specific profiles, provider status, and provider-related marketplace data.
services	Service creation, update, listing, and marketplace presentation logic.
orders	Customer-side service requests and order progression workflows.
bids	Offer and bid submission, review, and response handling between providers and users.
chat	Message exchange and conversation support between marketplace participants.
complaints and reviews	Trust-building functions such as complaints, dispute-related actions, and user feedback records.
recommendations	Recommendation-oriented processing for customer problems, suggested solutions, and provider matching
reviews	Ratings, reviews, and feedback related to completed marketplace interactions
notifications and admin_panel	System notifications, administrative monitoring, moderation-related supervision, and control operations.
knowledge	Provider-owned knowledge sources, documents, chunks, ingestion jobs, embeddings, visibility, and retrieval.
customer_assistant	Authenticated provider-specific questions, conversation sessions, grounded answers, insufficient-evidence handling, and citations.

7.3 API Implementation

Add Service Listing API

This section presents a simple implemented API from the Syrian Services Hub project. The selected API allows an authenticated service provider to create a new service listing on the platform.

API Name	Create Service Listing API
HTTP Method	POST
Endpoint	/api/v1/services/me/
Access	Authenticated provider only
Purpose	Create and store a new service listing for the logged-in provider

Request Processing Flow

- 1 The provider submits the service information through a POST request.
- 2 The API validates the submitted fields.

- 3 The service layer confirms that the logged-in user is a provider.
- 4 The new service record is saved and returned in the response.

1. Main API Route

File Location: backend/config/urls.py

PYTHON | ROUTING

```
path("api/v1/services/", include("apps.services.api.urls")),
```

Explanation: This line registers the services module in the main backend routing file. Therefore, every service-related endpoint begins with the /api/v1/services/ path prefix.

2. Service Creation Endpoint

File Location: backend/apps/services/api/urls.py

PYTHON | ENDPOINT

```
path("me/", MyServicesView.as_view(), name="services-me"),
```

Explanation: This route connects the /me/ path to MyServicesView. Combined with the main prefix, the final service creation endpoint is POST /api/v1/services/me/.3. Request Data Validation

File Location: backend/apps/services/api/serializers.py

PYTHON | SELECTED VALIDATION FIELDS

```
class ServiceCreateUpdateSerializer(serializers.Serializer):
    title = serializers.CharField(max_length=255)
    description = serializers.CharField()
    category = serializers.CharField(max_length=60)
    price = serializers.DecimalField(max_digits=10, decimal_places=2)
    price_type = serializers.ChoiceField(
        choices=["fixed", "hourly", "starting_at"]
    )
```

Explanation: The serializer checks the essential service information before creation. It validates the title, description, category, price, and allowed price type values. Optional fields such as duration and active status are omitted here for brevity.

4. API View Implementation

File Location: backend/apps/services/api/views.py

PYTHON | CREATE REQUEST HANDLER

```
class MyServicesView(APIView):
    permission_classes = [permissions.IsAuthenticated]

    def post(self, request):
        serializer = ServiceCreateUpdateSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        service = ServiceListingService.create_service(
            actor=request.user,
            attrs=serializer.validated_data
        )

        return success_response(
            data={"service": ServiceSerializer(service).data},
            message="Service created",
            status_code=201
        )
```

Explanation: The view accepts a POST request from an authenticated user, validates the request data, calls the service creation logic, and returns the newly created service with HTTP status code 201.

5. Business Rule and Database Creation

File Location: backend/apps/services/services/service_service.py

PYTHON | BUSINESS LOGIC

```
def create_service(*, actor, attrs):
    if actor.role != "provider":
        raise PermissionDeniedDomain("Only providers can create services.")

    provider = ProviderService.get_or_create_for_user(actor)
    normalized_attrs = dict(attrs)
    normalized_attrs["category"] = provider.category

    return ServiceRepository.create(provider=provider, **normalized_attrs)
```

Explanation: The business layer verifies that only providers can add services, creates the record through the repository layer, and uses the category stored in the provider profile before saving.

6. Example Request Body

JSON | REQUEST BODY

```
{
  "title": "Home Electrical Repair",
  "category": "Electrical",
  "price": "50000.00",
  "price_type": "fixed"
}
```

Explanation: The provider submits the main service details in JSON format.

7. Example Response Body

JSON | SUCCESS RESPONSE

```
{
  "success": true,
  "message": "Service created",
  "data": {
    "service": {
      "id": 1,
      "title": "Home Electrical Repair"
    }
  }
}
```

Explanation: The backend confirms successful creation and returns the created service data.

7.4 Implementation of the Data-Access Layer

The backend follows a layered organization in which persistence concerns are kept separate from request handling and higher-level business rules. The repository guidance states that persistence logic should remain in repositories and selectors, while API transport logic should remain in the presentation layer and business rules should remain in the service layer. This implementation choice supports cleaner domain boundaries and reduces the risk of mixing data-access code with workflow-specific logic. In practice, the data-access layer is responsible for retrieving, storing, and updating marketplace records such as users, services, orders, bids, chats, complaints, and reviews.

table 35 Data-access responsibilities in the implementation

Layer Element	Implementation Responsibility
Models and persistence structures	Represent stored entities and their relationships inside the PostgreSQL-backed backend system.
Repositories / selectors	Retrieve, filter, aggregate, and persist records required by the marketplace workflows.
Data transactions	Support reliable creation and update operations for service, order, bid, and interaction records.

7.5 Implementation of the Business-Logic Layer

The business-logic layer is a central part of the implementation because it coordinates platform rules independently from both the user interface and the persistence mechanisms. The repository standards explicitly state that business rules should be kept in the backend service layer. This means that core workflow decisions, permission checks, ownership validation, marketplace state transitions, and trust-related rules are handled through server-side services rather than scattered across controllers or UI code. Such a design is especially important for Syrian Services Hub because the platform supports multiple roles, service requests, bids, reviews, complaints, and administrative actions that must remain consistent across the system.

table 36 Representative business-logic responsibilities

Business Area	Examples of Implemented Logic
Authentication and access control	Role-based authorization, ownership checks, and protected access to domain operations.

Marketplace workflow control	Validation of service, order, and bid actions according to platform rules and user roles.
Trust and supervision	Complaint-related handling, review constraints, and administrative control over monitored processes.
Verification governance	Verification status review and administrative control over provider verification states, including revocation when required.
Ask Assistant orchestration	Validate the selected provider and user scope, rewrite or normalize the question when needed, retrieve evidence, generate a grounded response, reject unsupported answers, persist turns, and create citations.

7.6 User Interface Implementation

The frontend implementation is built with React and TypeScript and is organized as the client-facing layer of the platform. The repository describes the frontend as an existing React application that was incrementally stabilized and refactored, which means that the implementation process focused on preserving useful interface value while improving structure and backend integration. The user interface supports the main marketplace roles and workflows through pages and interactive components for service browsing, authentication, profile interaction, service requests, bids, and role-aware dashboards. In addition, the frontend was integrated with backend contracts through API adapters, helping the client application operate against the implemented server-side modules rather than mock-only flows.

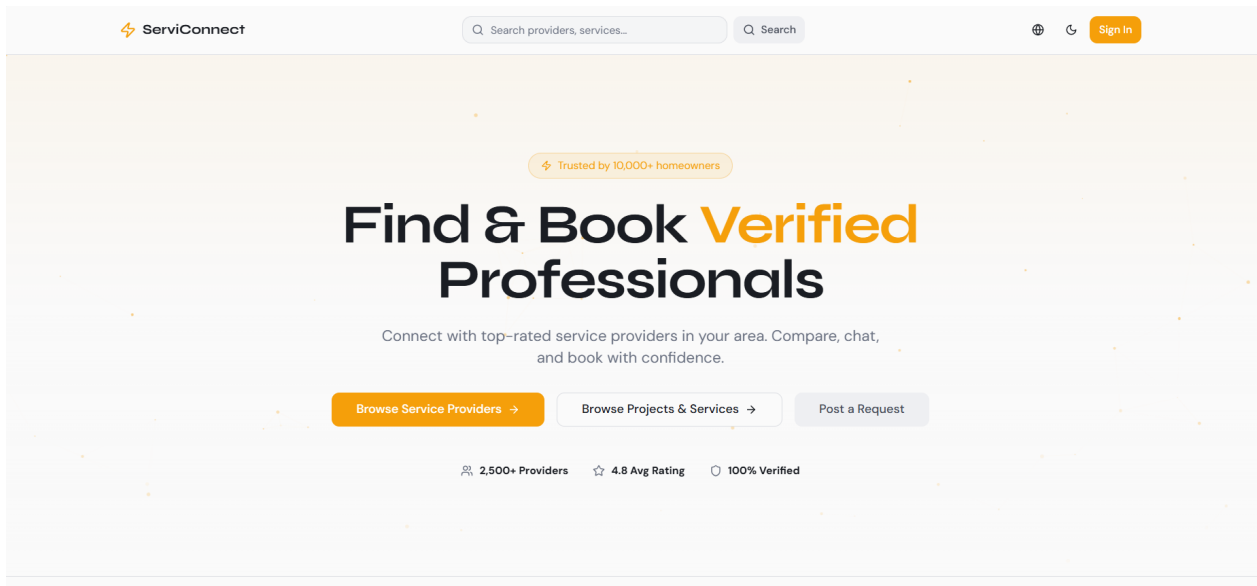


Figure 12 Placeholder for a main marketplace interface screenshot.

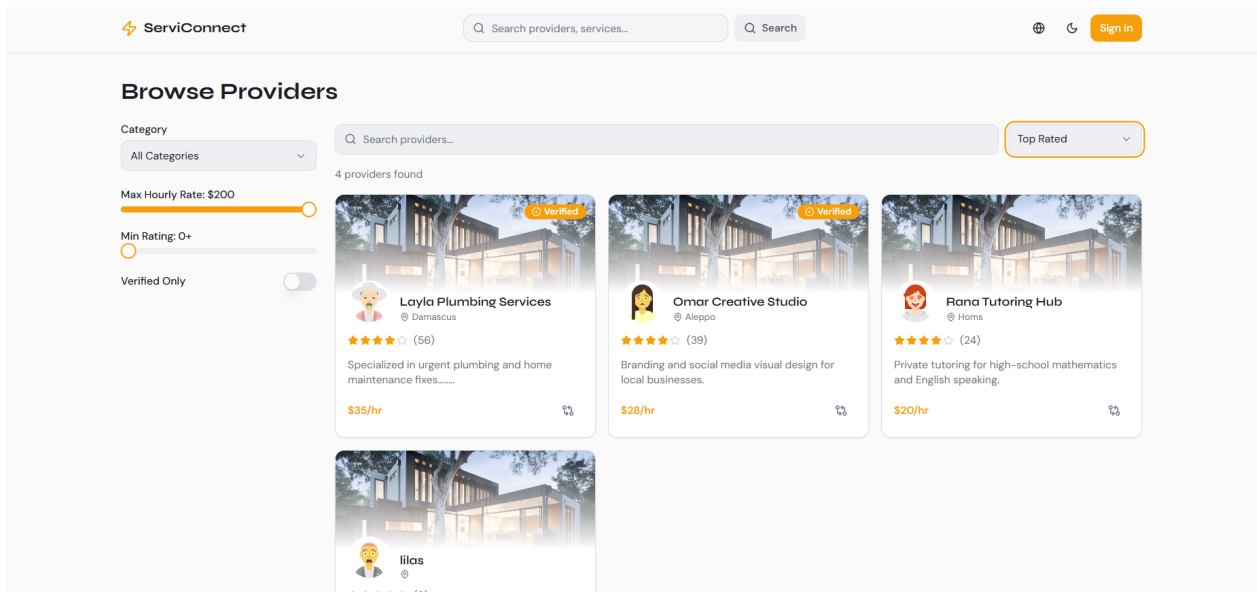


Figure 13 service-request interface screenshot

7.7 Implementation of the Intelligent Recommendation-Oriented Functionality

Syrian Services Hub includes an **intelligent recommendation-oriented feature** designed to assist users in identifying relevant services and suitable providers based on their submitted needs.

The current implementation primarily focuses on the core marketplace modules, authentication, frontend-backend integration, and workflow reliability. The recommendation capability is therefore implemented as an **incremental extension** aligned with the existing architecture and project requirements.

Its integration within the business logic layer allows the intelligent functionality to evolve independently while maintaining compatibility with the platform’s core services, data structures, and operational workflows.

table 37 Implementation positioning of the recommendation-oriented feature

Aspect	Implementation Interpretation
Input	A user-provided problem description or expressed service need.
Expected output	Suggested service direction and a suitable provider recommendation.
Architectural position	Implemented as an extensible logic component connected to marketplace data and workflow context.
Current implementation emphasis	Core marketplace modules and integration are primary; recommendation support is framed as the intelligent extension of the platform.

7.8 Implementation of Ask Assistant (RAG)

Ask Assistant is implemented independently from **AI Recommendation** through two backend domains: **knowledge** and **customer_assistant**. The knowledge domain manages provider-owned sources, content extraction, chunking, embeddings, and retrieval, while customer_assistant handles sessions, questions, answers, and citations.

The main endpoint, **POST /api/v1/customer-assistant/questions/**, authenticates users, validates provider context, processes questions, and returns grounded answers with citations and session data. Providers can also upload, archive, and reindex their authorized knowledge sources.

During ingestion, documents are validated, extracted, divided into overlapping chunks, and converted into vector embeddings.

Chapter Eight: Testing and Evaluation

This chapter evaluates the **functionality, reliability, usability, and security** of Syrian Services Hub. Testing focuses on the main marketplace workflows, role-based access, communication, administrative operations, and intelligent features to confirm that system requirements are implemented as observable and testable behavior.

8.1 Test Plan

The test plan defines the testing scope, objectives, environment, responsibilities, and acceptance criteria. It emphasizes **scenario-based testing** of key workflows, including browsing, account operations, service management, requests, offers, messaging, ratings, complaints, verification, and recommendation functionality.

Ask Assistant is tested separately from AI Recommendation, focusing on provider authorization, source visibility, ingestion status, retrieval relevance, citation accuracy, answer grounding, conversation ownership, and correct handling of insufficient evidence.

table 38 Test plan overview

Item	Description
Testing Objective	Verify that the main system workflows operate correctly and satisfy the documented functional and non-functional requirements.
Testing Scope	Accounts, services, service requests, offers, messaging, reviews, complaints, verification, dashboards, and recommendation functionality.
Testing Approach	Manual scenario-based testing supported by API validation for backend endpoints and workflow behavior.
Testing Environment	Local development environment using the implemented frontend, backend, database, and API testing tools.
Testing Tools	Browser-based testing, Postman for API checks, development logs, and database inspection when needed.
Acceptance Criteria	A workflow is accepted when it produces the expected result, prevents invalid behavior, respects permissions, and keeps stored data consistent.
RAG Testing Scope	Knowledge-source ingestion, provider ownership, active-source filtering, vector retrieval, Ask Assistant API, citations, session history, and insufficient-evidence handling.

8.2 Types of Testing

Several testing types are relevant to Syrian Services Hub because the platform combines user-facing interaction, backend services, database operations, and access-control rules. The testing strategy therefore includes functional testing for feature correctness, integration testing for communication between frontend and backend layers, system testing for full workflow validation, acceptance testing for user-oriented scenarios, security testing for permissions and protected operations, performance-oriented checks for responsiveness, and usability review for interface clarity.

table 39 Types of testing applied to Syrian Services Hub

Testing Type	Purpose	Examples in the Project
Functional Testing	Validate individual functions against expected behavior.	Login, service creation, request posting, offer submission, complaint creation.
Integration Testing	Verify interaction between frontend, API layer, backend logic, and database.	Submitting a request from the interface and storing it correctly in the database.
System Testing	Validate complete end-to-end workflows.	User posts a request; provider submits an offer; user accepts or rejects the offer.
Acceptance Testing	Check that key workflows satisfy stakeholder needs.	User can find a relevant service or provider using browsing or recommendation.
Security Testing	Verify role-based access and protected operations.	Only Admin can manage complaints or revoke provider verification.
Performance Checks	Observe responsiveness during common operations.	Browsing services, filtering, opening dashboards, and submitting forms.
Usability Review	Evaluate clarity, navigation, and user experience.	Checking whether forms, statuses, and actions are understandable.
RAG Grounding and Permission Testing	Verify that answers use only authorized active evidence and that every supported answer is traceable.	A provider-specific question returns a cited answer; private, archived, unrelated, or weak evidence is excluded and produces a safe response.

8.3 Test Diagrams

Test diagrams are used to summarize the testing workflow and show how major scenarios move from requirement selection to execution and result recording. They help clarify the relationship between requirements, test cases, expected results, and final evaluation. Since the actual detailed diagrams are maintained separately as project figures, this section reserves the required locations for inserting the final testing diagrams without increasing the working file size.

8.3.1 Test Case Diagram

The test case diagram should represent the main testable workflows of the platform, especially the scenarios that involve multiple actors or critical system behavior. These include service browsing, service request creation, offer submission, provider verification handling, complaint management, and recommendation generation.

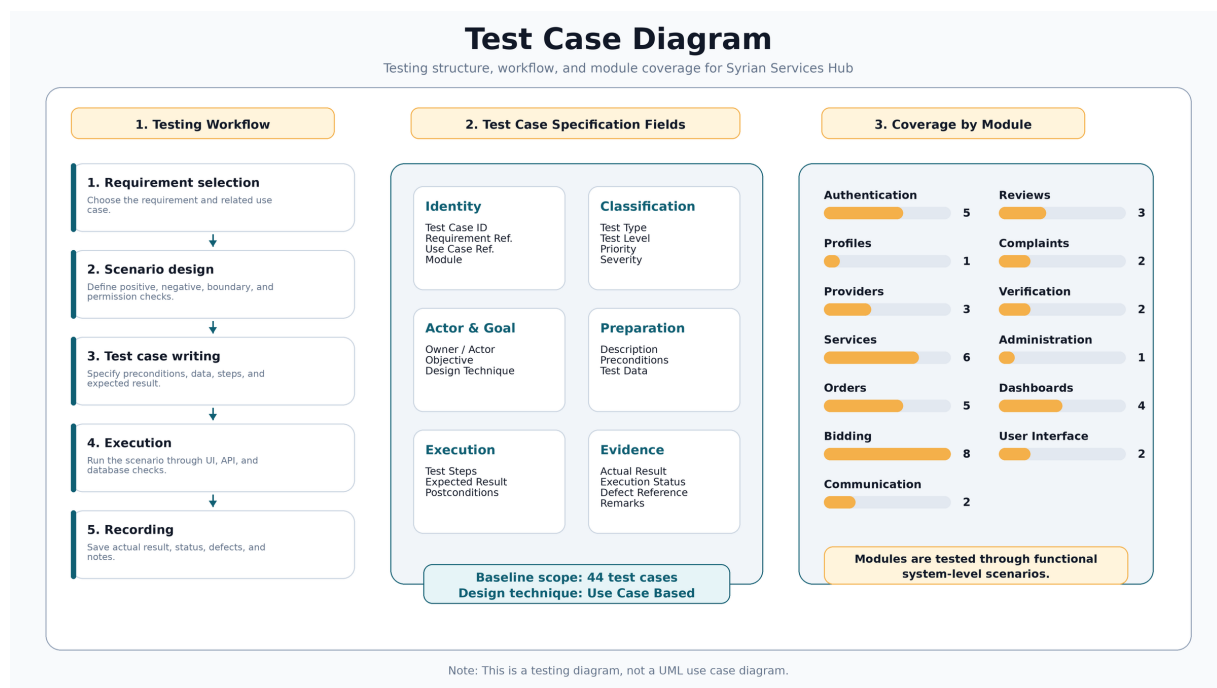


Figure 8.1: Test case diagram for the main testable workflows of Syrian Services Hub.

8.3.2 Test Results Diagram

The test results diagram should summarize the status of executed test cases, such as passed, failed, or pending items. It can be represented as a simple chart after the final execution results

are confirmed. This keeps the evaluation section visually clear while preserving the detailed results in the testing tables.

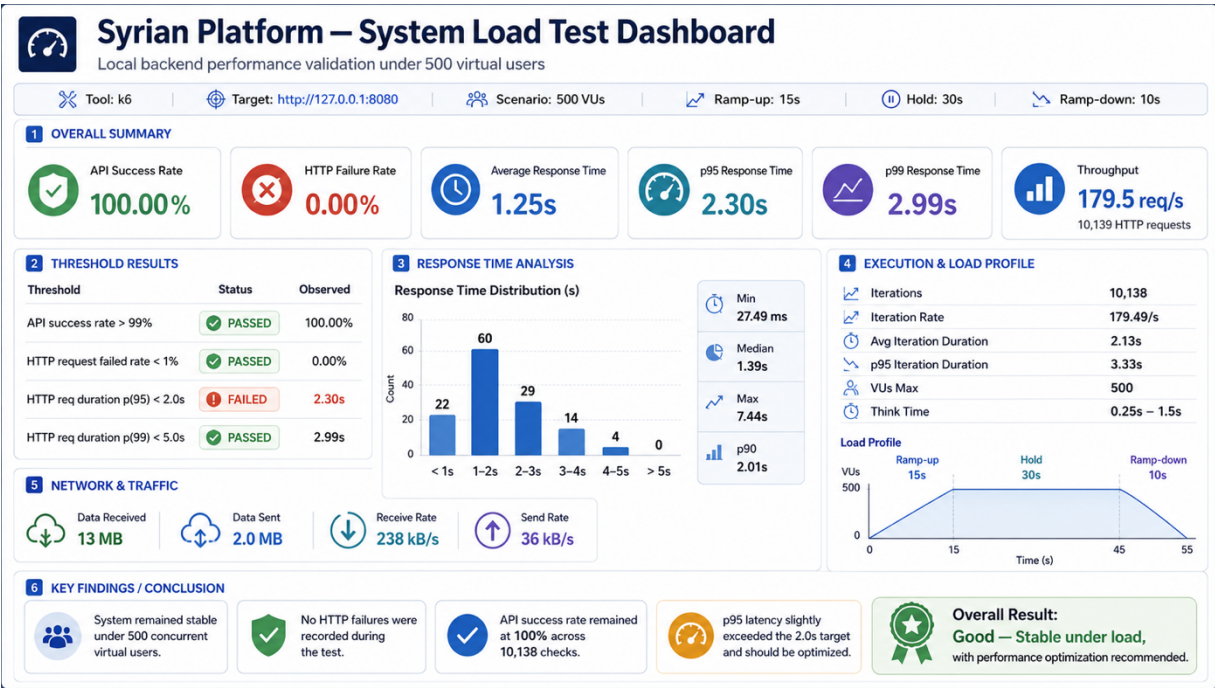


Figure 8.2: Summary diagram of testing results and validation status.

8.4 Test Results and Analysis

The following test cases summarize the main validation scenarios for Syrian Services Hub. The selected cases focus on the core platform behavior and the most important stakeholder workflows. They also include the intelligent recommendation feature and administrative verification control because these parts are essential to the system scope and should be verified clearly. The results are structured around expected outcomes and acceptance status, which makes the testing process traceable to the requirements and use case descriptions defined earlier.

table 40 Main test cases and validation results add services

Test Case ID	TC-SERV-001	Requirement Ref.	SERV-001
Use Case Ref.	UC-10	Module	Services
Title	Add Service	Owner / Actor	Provider

Test Type	Functional / Positive	Test Level	System
Priority	High	Severity	High
Risk Level	High	Automation Candidate	Medium
Objective	Verify that the system satisfies the Add Service requirement according to the defined main scenario.	Design Technique	Use Case Based
Description		Allows the service provider to add a new service that includes the service name, description, price, and execution duration.	
Pre-conditions		- The required actor is logged in. - The Add Service page is available.	
Test Data / Input		- Service Name: Air Conditioner Maintenance - Price: 150 - Execution Duration: Two days	
Test Steps		- Open the screen or section related to the "Add Service" feature. - Enter or select valid test data when needed. - Perform the main action related to the feature. - Review the result displayed to the user or actor. - Verify that the change is saved or that data is retrieved from the database according to the expected behavior.	
Expected Result		- A new service record linked to the service provider is created. - No unexpected error messages or behavior that contradicts the scenario appear. - The interface or data is updated according to the performed operation.	
Post-conditions		A new service record linked to the service provider is created.	
Exception Notes		There are no additional exception notes within this basic case; other scenarios are covered in later test cases.	
Actual Result	The test case TC-SERV-001 for "Add Service" was executed successfully, and the actual result matched the expected result with no functional errors.	Execution Status	Pass
Defect Reference	N/A	Executed By	QA Tester

Reviewed By	QA Lead	Date Created	2026-03-10
Execution Date	2026-03-10	Remarks / Notes	These fields were filled as an initial execution sample and can be updated during actual execution.

table 41 Main test cases and validation results submit offer

Test Case ID	TC-SERV-008	Requirement Ref.	SERV-008
Use Case Ref.	UC-17	Module	Orders
Title	Submit Offer on Service Request	Owner / Actor	Provider
Test Type	Functional / Positive	Test Level	System
Priority	High	Severity	High
Risk Level	High	Automation Candidate	Medium
Objective	Verify that the system satisfies the Submit Offer on Service Request requirement according to the defined main scenario.	Design Technique	Use Case Based
Description		Allows the service provider to respond to the service request sent by the user by submitting an offer that includes the price, duration, or execution details.	
Pre-conditions		- The required actor is logged in. - There is an incoming service request with Pending status.	
Test Data / Input		- Proposed Price: 120 - Proposed Duration: 2 days - Execution Details: Visit and inspection	
Test Steps		- Open the screen or section related to the "Submit Offer on Service Request" feature. - Enter or select valid test data when needed. - Perform the main action related to the feature. - Review the result displayed to the user or actor. - Verify that the change is saved or that data is retrieved from the database according to the expected behavior.	

Expected Result		• A new offer record linked to the service request is created, and the request status is updated to indicate that it has been answered with an offer. • No unexpected error messages or behavior that contradicts the scenario appear. • The interface or data is updated according to the performed operation. • In the exceptional scenario: the offer data is invalid or the request is no longer in Pending status.	
Post-conditions		A new offer record linked to the service request is created, and the request status is updated to indicate that it has been answered with an offer.	
Exception Notes		The offer data is invalid or the request is no longer in Pending status.	
Actual Result	The test case TC-SERV-008 for "Submit Offer on Service Request" was executed successfully, and the actual result matched the expected result with no functional errors.	Execution Status	Pass
Defect Reference	N/A	Executed By	QA Tester
Reviewed By	QA Lead	Date Created	2026-03-10
Execution Date	2026-03-10	Remarks / Notes	These fields were filled as an initial execution sample and can be updated during actual execution.

table 42 Main test cases and validation results post new request

Test Case ID	TC-BID-001	Requirement Ref.	BID-001
Use Case Ref.	UC-21	Module	Bidding
Title	Post New Request	Owner / Actor	User
Test Type	Functional / Positive	Test Level	System
Priority	High	Severity	High
Risk Level	High	Automation Candidate	Medium

Objective	Verify that the system satisfies the Post New Request requirement according to the defined main scenario.	Design Technique	Use Case Based
Description	Allows the user to post a service request with the service description and expected budget.		
Pre-conditions	- The required actor is logged in. - The Post New Request page is available.		
Test Data / Input	- Description: Electrical network installation - Expected Budget: 500		
Test Steps	- Open the screen or section related to the "Post New Request" feature. - Enter or select valid test data when needed. - Perform the main action related to the feature. - Review the result displayed to the user or actor. - Verify that the change is saved or that data is retrieved from the database according to the expected behavior.		
Expected Result	- A new service request record is created in the database. - No unexpected error messages or behavior that contradicts the scenario appear. - The interface or data is updated according to the performed operation.		
Post-conditions	A new service request record is created in the database.		
Exception Notes	There are no additional exception notes within this basic case; other scenarios are covered in later test cases.		
Actual Result	The test case TC-BID-001 for "Post New Request" was executed successfully, and the actual result matched the expected result with no functional errors.	Execution Status	Pass
Defect Reference	N/A	Executed By	QA Tester
Reviewed By	QA Lead	Date Created	2026-03-10
Execution Date	2026-03-10	Remarks / Notes	These fields were filled as an initial execution sample and can be updated during actual execution.

8.5 Quality Metrics

Quality metrics are used to evaluate whether the platform satisfies the non-functional expectations defined earlier in the requirements chapter. In this project, the most relevant metrics are usability, response behavior, security, reliability, traceability, and recommendation relevance. Since the project is an academic software engineering system, the evaluation focuses on practical indicators that can be checked through manual testing, workflow review, and implementation inspection.

table 45 Quality metrics used in the evaluation

Quality Aspect	Metric / Indicator	Evaluation Method	Target
Usability	Clarity of navigation, forms, and status messages.	Manual walkthrough of key workflows.	Users can complete the main actions without confusion.
Performance	Observed response during common operations.	Browsing, filtering, form submission, and dashboard access.	No blocking delay during normal local testing.
Security	Role-based access and protected administrative actions.	Permission checks and unauthorized-action attempts.	Restricted actions remain accessible only to allowed roles.
Reliability	Consistency of stored records and workflow transitions.	Database and UI state verification after actions.	Created records remain linked to the correct user, request, or provider.
Traceability	Connection between requirements and test cases.	RTM and testing tables.	Each critical requirement has at least one related test case.
Recommendation Relevance	Returned suggestions match the entered problem context.	Manual review of recommendation outputs.	Suggested solutions and providers are meaningfully related to the user problem.
RAG Groundedness	Citation coverage, evidence support, and safe refusal rate.	Review answer-citation pairs and execute unsupported-question scenarios.	Supported factual answers are cited and evidence-based;

			unsupported answers are not fabricated.
--	--	--	---

8.6 Usability Testing

Usability testing focuses on whether the main actors can understand and complete the platform workflows in a clear and efficient way. For Syrian Services Hub, usability is especially important because the platform serves different roles with different expectations: visitors need simple browsing, users need clear request and recommendation paths, providers need organized service and offer management, and administrators need understandable control workflows. The usability review therefore focused on the clarity of page structure, the visibility of actions, the simplicity of forms, and the understandability of status changes.

table 46 Usability review observations

Area	Observation	Improvement Direction
Service browsing	Users should be able to reach services quickly through clear navigation.	Keep categories, cards, and filters visible and simple.
Request posting	The service-demand form should be direct and easy to complete.	Use concise labels, validation messages, and clear submission feedback.
Offer management	Offer status should be understandable for both user and provider.	Use consistent labels such as pending, accepted, rejected, or completed.
Recommendation page	The user should understand what to enter and what the system will return.	Explain that the system returns suggested solutions and a suitable provider.
Administrative actions	Sensitive actions such as verification review and revocation should be explicit.	Use confirmation messages and clear status indicators.
Ask Assistant panel	The user should understand the selected provider context, the evidence-based nature of the answer, and the meaning of citations or insufficient evidence.	Show provider context, question input, answer, source links/excerpts, loading state, and a clear insufficient-evidence message.

The usability evaluation shows that the platform should remain simple and role-oriented. The most important usability requirement is to prevent users from feeling that the system has too many unrelated options. Instead, each actor should see clear actions related to their role.

This supports the broader objective of Syrian Services Hub as a structured and accessible service marketplace.

8.7 Evaluation Summary

The testing and evaluation process demonstrates that Syrian Services Hub can be validated through a structured set of functional, integration, security, usability, and quality-oriented checks. The selected test cases cover the most important workflows, including browsing, account access, service management, service requests, offers, messaging, complaints, verification control, and recommendation support. The evaluation confirms that the system behavior can be tested against clear criteria and that the major requirements are connected to practical validation scenarios. The remaining visual testing artifacts, such as the test case diagram and results diagram, can be inserted later without changing the structure or content of this chapter.

Section Five: Results and Recommendations

Chapter Nine: Results and Analysis

This chapter presents the main results achieved by Syrian Services Hub and analyzes how the developed system responds to the project objectives. It summarizes the practical output of the platform, compares the proposed solution with existing service marketplace platforms, evaluates the degree to which the project goals were achieved, and discusses the main strengths, limitations, and performance indicators of the system.

9.1 System Results

The implementation of **Syrian Services Hub** resulted in a structured digital marketplace supporting visitors, users, service providers, and administrators through role-based access. Core functions include service browsing, service requests, offer management, messaging, provider profiles, complaints, verification, and administrative monitoring.

The platform also incorporates an **intelligent recommendation feature** that analyzes a user's problem description to suggest relevant solutions and suitable service providers, helping users translate their needs into more informed service decisions.

In addition, **Ask Assistant** provides authenticated, provider-specific question answering using approved knowledge sources. Grounded answers are stored with citations, giving users transparent and traceable information about providers and services.

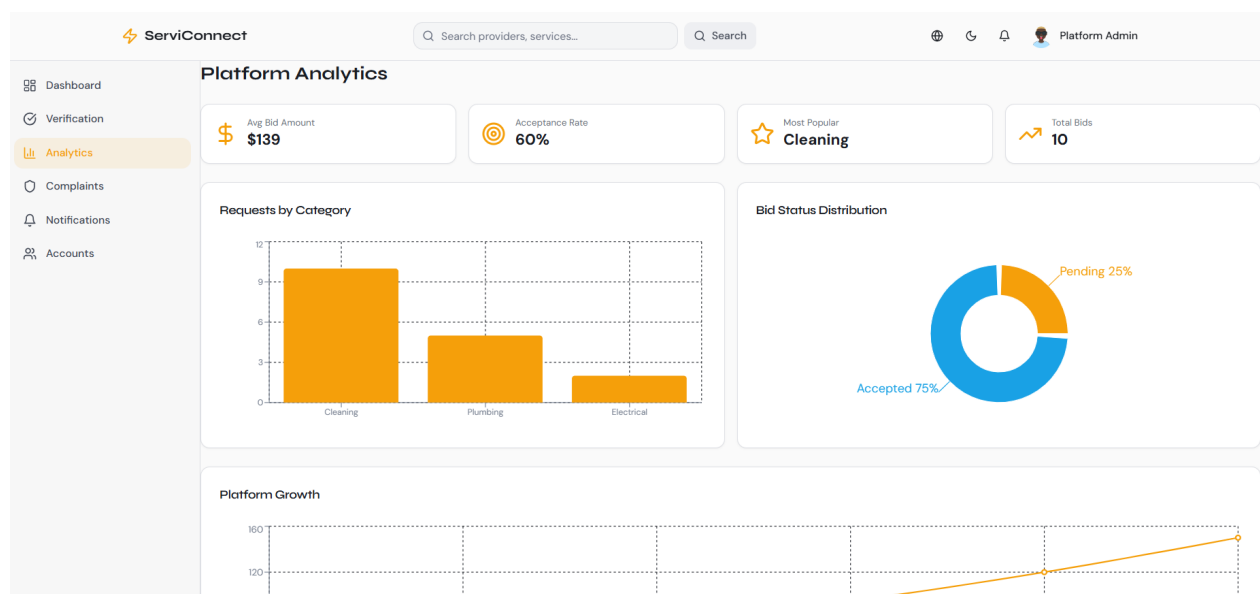


Figure 14 dashboard of Syrian Services Hub.

9.2 Comparative Analysis with Existing Systems

The benchmark study showed that existing platforms differ in their operating models. Some platforms focus mainly on ready-made services, while others focus on open service requests and competitive offers. Syrian Services Hub combines both models in one platform and adds recommendation-oriented support to improve service discovery and decision-making.

table 47 Comparative analysis between Syrian Services Hub and selected reference platforms.

Criterion	Khamsat / Fiverr	Mostaq / Upwork	Syrian Services Hub
Service model	Strong ready-made service model	Strong request and offer model	Hybrid model: services + requests + offers
User decision support	Mostly based on browsing and ratings	Based on proposals, profiles, and filters	Recommendation support plus provider comparison logic
Local adaptation	General marketplace model	General freelance/project model	Adapted to a local service marketplace context
Trust features	Ratings and profile indicators	Verification, proposals, messages	Ratings, complaints, verification, and admin control
Smart feature	Limited or implicit	Limited or platform-specific	Problem-based recommendation for solutions and providers
Knowledge-grounded customer Q&A	Generally unavailable as provider-owned cited Q&A	Generally not a core marketplace workflow	Ask Assistant retrieves provider-owned evidence and returns cited answers or an insufficient-evidence response

9.3 Achievement of Project Objectives

The achieved results can be evaluated by comparing the implemented and designed system behavior with the objectives defined at the beginning of the project. The platform succeeded in addressing the major functional areas required for a smart service marketplace, including service discovery, service request posting, offer submission, interaction, trust-related functions, and recommendation support.

table 48 Achievement of the main project objectives.

Objective	Result / Evidence	Status
Connect consumers with service providers	The system supports users, providers, services, requests, and offers.	Achieved
Support hybrid marketplace workflows	The platform includes ready-made services and open service-demand advertisements.	Achieved
Improve trust and supervision	Ratings, complaints, verification, verification revocation, and admin monitoring are supported.	Achieved
Provide intelligent recommendation support	The user enters a problem and receives suggested solutions and a recommended provider.	Achieved
Support maintainable system design	The system follows a client-server and three-tier architectural structure.	Achieved
Provide grounded customer Q&A through Ask Assistant	The system separates RAG from recommendation, retrieves authorized provider evidence, returns citations, and handles insufficient evidence safely.	Achieved

9.4 Strengths and Limitations

The project has several strengths that make it suitable as an academic software engineering solution. It combines multiple service-marketplace workflows in a unified platform, supports several user roles, and includes trust-related mechanisms that improve platform reliability. The recommendation feature also adds a smart dimension by helping users move from a problem description to suggested solutions and a suitable provider. However, the project also has limitations, especially because it was developed within an academic timeframe. Some advanced production-level features, such as full payment integration, large-scale deployment optimization, and extensive real-user evaluation, can be considered future improvements rather than core requirements of the current version.

table 49 Summary of project strengths and limitations.

Strengths	Limitations
Hybrid service marketplace model combining services, requests, and offers.	No complete online payment workflow in the current academic scope.
Clear role-based structure for visitor, user, provider, and admin.	Large-scale production deployment remains a future enhancement.
Trust mechanisms through ratings, complaints, verification, and admin control.	Recommendation quality can be improved later with more data and advanced models.
Recommendation feature supports users with suggested solutions and providers.	Usability testing with a large number of real users is still limited.
Separate RAG architecture with provider-owned knowledge, permission-aware retrieval, citations, and explicit insufficient-evidence handling.	Answer quality depends on the completeness, freshness, and clarity of provider knowledge sources and requires broader real-world evaluation.

9.5 Key Performance Indicators

Key performance indicators are used to evaluate whether the system provides acceptable quality from functional, usability, and operational perspectives. Since the project is developed in an academic context, the indicators are used mainly to guide evaluation and future improvement rather than to claim large-scale production performance.

table 50 Suggested KPIs for evaluating Syrian Services Hub.

KPI	Evaluation Focus	Expected Direction
Response time	Speed of browsing, searching, and submitting requests or offers	Fast and acceptable for normal academic testing
Requirement coverage	Coverage of functional requirements by design and test cases	High coverage for core requirements
Recommendation relevance	Suitability of suggested solutions and providers to the entered problem	Relevant recommendations for common problem descriptions
Usability	Clarity of main user paths and ease of interaction	Simple and understandable workflows
Reliability	Correct handling of roles, data, complaints, and verification status	Consistent behavior across main scenarios
RAG grounded-answer rate	Percentage of supported answers with valid citations and percentage of unsupported questions safely rejected	High citation coverage with no cross-provider evidence leakage

Chapter Ten: Conclusion and Recommendations

This chapter concludes the thesis by summarizing the main achievements, challenges, future improvements, and recommendations of **Syrian Services Hub**. It reflects the overall value of the platform as a structured digital marketplace supported by intelligent recommendation and RAG-based assistance.

10.1 Project Summary and Main Achievements

Syrian Services Hub was developed as a smart digital service marketplace connecting consumers and service providers through a unified platform. It supports service browsing, requests, offers, messaging, and role-based access for visitors, users, providers, and administrators.

A key achievement is the **hybrid marketplace model**, which combines direct service publishing with open service requests and competitive offers. Trust mechanisms include ratings, complaints, verification, administrative supervision, and verification revocation.

Table 51. Summary of Main Project Achievements

Achievement Area	Summary
Marketplace Structure	Supports ready-made services and open service requests.
Provider Interaction	Providers can publish services and submit offers.
Trust and Supervision	Ratings, complaints, verification, and administrative control improve reliability.
AI Recommendation	Suggests relevant solutions and suitable providers.
Engineering Documentation	Includes structured requirements, design, implementation, and testing.
Ask Assistant (RAG)	Provides provider-grounded answers with citations.

10.2 Difficulties and Challenges

The project faced challenges related to its broad functional scope, integration between modules, evolving requirements, documentation consistency, and limited academic time.

The intelligent features introduced additional complexity. AI Recommendation had to remain aligned with marketplace needs, while **Ask Assistant** required careful handling of ingestion, chunking, vector retrieval, authorization, citation integrity, and unsupported responses. These challenges were addressed through modular design, incremental development, permission-first retrieval, and grounding-focused rules.

Table 52. Main Project Challenges and Responses

Challenge	Project Response
Large functional scope	Features were prioritized and organized into modules.
Requirement refinement	Requirements and use cases were continuously reviewed.
Recommendation integration	Recommendations were linked directly to user needs.
Documentation consistency	Analysis, design, implementation, and testing were aligned progressively.
Time constraints	Development was divided into manageable phases.
RAG grounding and authorization	Retrieval was permission-controlled, citations were stored, and unsupported answers were rejected.

10.3 Future Improvements

Future development can focus on improving intelligence, usability, and scalability. AI Recommendation may be enhanced through larger datasets, machine learning, behavior analysis, and personalized ranking. Provider comparison can also be improved using ratings, response time, pricing, category, verification status, and activity history.

Additional improvements may include **mobile applications, stronger notifications, multilingual support, advanced analytics, scalable deployment, performance optimization, security auditing, and operational monitoring.**

Table 53. Suggested Future Improvements

Improvement	Expected Benefit
Advanced recommendation models	Improve recommendation relevance.

Enhanced provider comparison	Support better user decisions.
Mobile application support	Increase accessibility.
Advanced analytics	Improve monitoring and decision-making.
Performance and security hardening	Increase production readiness.

10.4 Recommendations

Future development should maintain a **modular and requirement-driven approach**, keeping core marketplace workflows stable before introducing additional complexity. Intelligent features should remain closely connected to practical user needs.

The project should also preserve **traceability between requirements, design, implementation, and testing**, while continuing to strengthen verification, ratings, complaints, and administrative supervision. Before production deployment, real-user testing, performance monitoring, and security assessment are recommended.

Table 54. Final Recommendations

Recommendation	Purpose
Keep core workflows simple and reliable.	Protect the main user journey.
Improve recommendation quality gradually.	Maintain useful and relevant intelligent support.
Preserve lifecycle traceability.	Support maintenance and consistency.
Strengthen trust mechanisms.	Increase confidence between users and providers.
Conduct real-user validation.	Verify usability, performance, and acceptance.
Keep AI Recommendation and Ask Assistant separate.	Maintain clear responsibility between matching and factual question answering.

Section Six:

Appendices and

References

Appendices

The appendices provide supporting material for the analysis, design, implementation, and testing activities discussed in the thesis. Detailed diagrams and screenshots may be inserted in their corresponding locations or attached in the appendices when preparing the final print version.

Appendix A: Detailed Use Case Diagrams.

Figure 15 Main Use Case Diagram for Syrian Services Hub

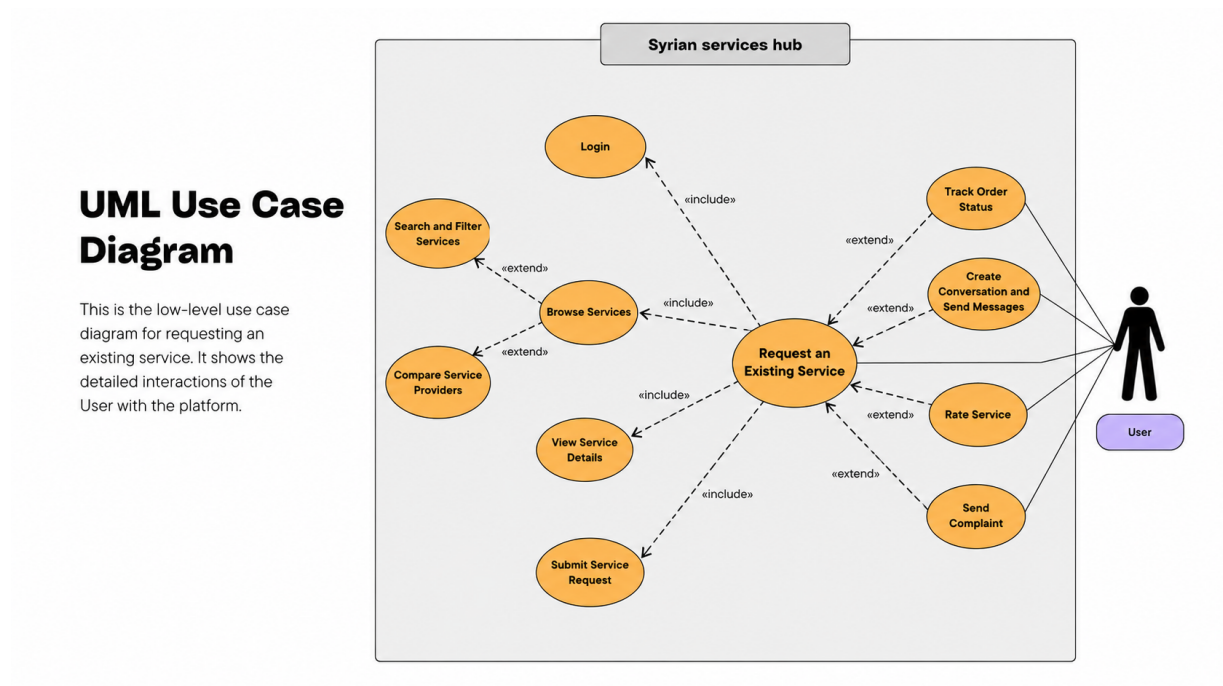


Figure 16 use cae request an exasting service

UML Use Case Diagram

This is the low-level use case diagram for the core use case "Post a Service Request Advertisement". It shows the detailed interactions of the User with the platform to create, manage, and process a service request advertisement.

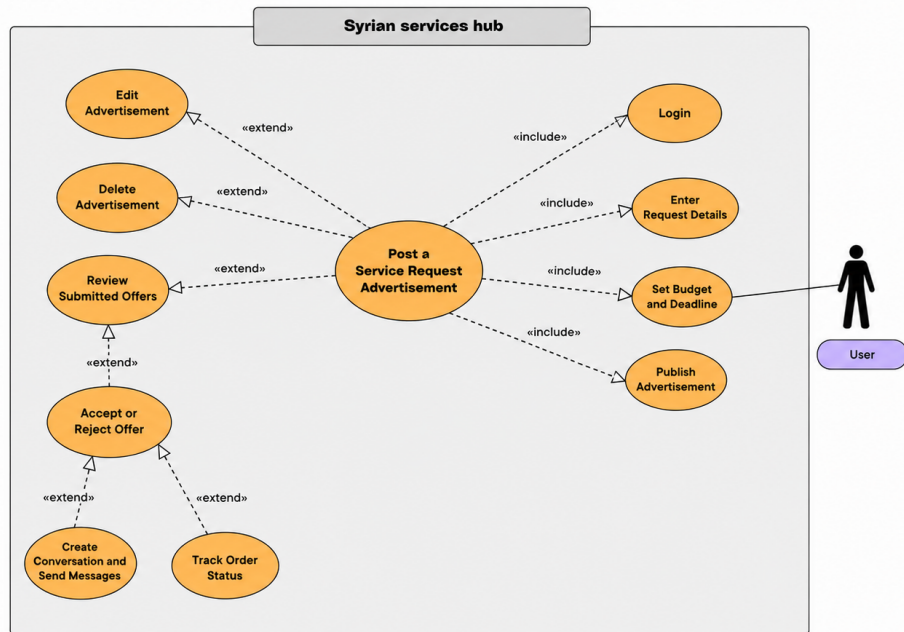


Figure 17 use cae post new request

UML Use Case Diagram

This is the low-level Use Case Diagram for submitting an offer on a service request advertisement. It illustrates the detailed interactions of the Service Provider with the platform, including mandatory steps (included use cases) and optional behaviors (extended use cases).

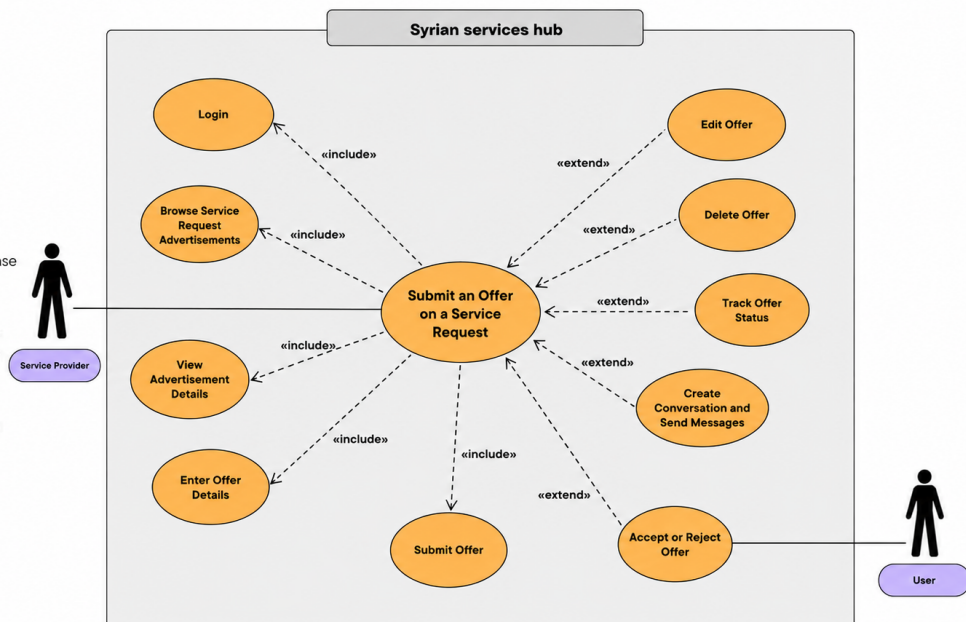
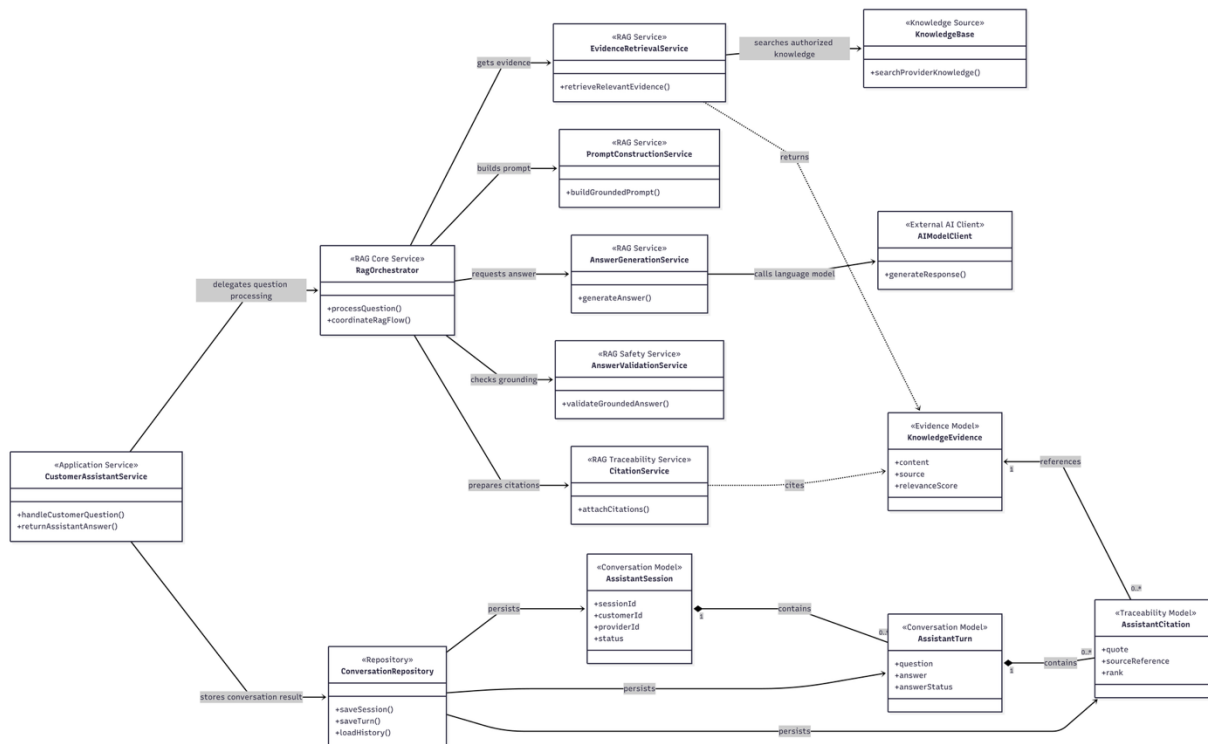


Figure 18 use case submite an offer

Appendix D: Ask Assistant (RAG) Technical Details



References

The following references include the official sources used in the benchmark study, the technical documentation related to the implementation stack, and the project repository used to verify implementation details. A single numbered reference style is used for consistency.

- [1] Khamsat Support Center, FAQ and profile settings documentation. Available: <https://support.khamsat.com/>
- [2] Mostaq Support Center, FAQ, completion rate, and offer package documentation. Available: <https://support.mostaql.com/>
- [3] Fiverr Help Center, Creating a Gig and seller workflow documentation. Available: <https://help.fiverr.com/>
- [4] Upwork Help Center, Project Catalog, proposals, direct messages, verification, and Trust and Safety resources. Available: <https://support.upwork.com/>
- [5] Freelancer Support Center, milestone payments, KYC, and dashboard documentation. Available: <https://www.freelancer.com/support/>
- [6] PeoplePerHour Support Center, escrow and project protection documentation. Available: <https://support.peopleperhour.com/>
- [7] React Documentation. Available: <https://react.dev/>
- [8] TypeScript Documentation. Available: <https://www.typescriptlang.org/docs/>
- [9] Django Documentation. Available: <https://docs.djangoproject.com/>
- [10] Django REST Framework Documentation. Available: <https://www.django-rest-framework.org/>
- [11] PostgreSQL Documentation. Available: <https://www.postgresql.org/docs/>
- [12] Vite Documentation. Available: <https://vite.dev/guide/>
- [13] Postman Learning Center. Available: <https://learning.postman.com/>
- [14] GitHub Documentation. Available: <https://docs.github.com/>
- [15] Syrian Services Hub project repository. Available: <https://github.com/lilasnkl/Syrian>
- [17] OpenAI API Documentation, Embeddings and Responses API. Available: <https://platform.openai.com/docs/>